

OPERATING SYSTEMS

UE24CS242B



Storage Management

Pavan A C

Department of Computer Science & Engineering

OPERATING SYSTEMS



File System

Pavan A C

Department of Computer Science & Engineering

OPERATING SYSTEMS

Slides Credits for all the PPTs of this course

- The slides/diagrams in this course are an **adaptation, combination, and enhancement** of material from the following resources and persons:

1. Slides of Operating System Concepts, Abraham Silberschatz, Peter Baer Galvin, Greg Gagne - 9th edition 2013 and some slides from 10th edition 2018
2. Some conceptual text and diagram from Operating Systems - Internals and Design Principles, William Stallings, 9th edition 2018
3. Some presentation transcripts from A. Frank – P. Weisberg
4. Some conceptual text from Operating Systems: Three Easy Pieces, Remzi Arpaci-Dusseau, Andrea Arpaci Dusseau



- Computers can store information on various storage media, such as magnetic disks, magnetic tapes, and optical disks.
- File is logical storage whereas disks are physical storage unit
- What is a File?
 - Collection of related information
 - Data cannot be written to disk unless they are in files
 - Data can be Numeric, alphabetic, alphanumeric, binary
- A file is a sequence of bits, bytes, lines, or records, the meaning of which is defined by the file's creator and user
- Contents of file can be source or executable programs, numeric or text data, photos, music, video, and so on.

OPERATING SYSTEMS

File Attributes

Files are named and are referred by their names by the human users.

Attributes of files

- **Name** – only information kept in human-readable form
- **Identifier** – unique tag (number) identifies file within file system
- **Type** – needed for systems that support different types
- **Location** – pointer to file location on device
- **Size** – current file size
- **Protection** – controls who can do reading, writing, executing
- **Time, date, and user identification** – data for protection, security, and usage monitoring
- Information about files are kept in the directory structure, which is maintained on the disk.



File is an abstract data type.

File Operations

- **Create**
- **Write** – Use the system call to write to a file. A write pointer points to the location in the file, where the next write is to take place.
- **Read** – – Use the system call to write to a file. A read pointer points to the location in the file from where the next read is to take place
- **Reposition within file** - Repositioning within a file need not involve any actual I/O. It is done with system call lseek. Repositioning is also called seek to location in a file
- **Delete:** Use the system call to delete/remove a file.
 - File to be deleted is searched in the directory.
 - Release the memory allocated after deletion

- **Truncate:** The user may want to erase the contents of a file but keep its attributes.
 - Instead of deleting the file and then recreating it, use the function that sets the size of file to zero.
- **Appending** to the end of a existing file
- ***Open a file*** – use a system call `open()`
 - **Open-file table:** tracks open files
 - **File pointer:** pointer to last read/write location, per process that has the file open
 - **File-open count:** counter of number of times a file is open – to allow removal of data from open-file table when last processes closes it
- ***Close:*** *When a file is actively not used close it.*

OPERATING SYSTEMS

File Operations

- Several pieces of information are associated with an open file.
 - File pointer
 - File-open count.
 - Disk location of the file.
 - Access rights



- Provided by some operating systems and file systems
 - Similar to reader-writer locks
 - **Shared lock** similar to reader lock – several processes can acquire concurrently
 - **Exclusive lock** similar to writer lock
- OS mediates access to a file
- Mandatory or advisory:
 - **Mandatory** – access is denied depending on locks held and requested
 - **Advisory** – processes can find status of locks and decide what to do

OPERATING SYSTEMS

File Types – Name, Extension



PES
UNIVERSITY

CELEBRATING 50 YEARS

file type	usual extension	function
executable	exe, com, bin or none	ready-to-run machine-language program
object	obj, o	compiled, machine language, not linked
source code	c, cc, java, pas, asm, a	source code in various languages
batch	bat, sh	commands to the command interpreter
text	txt, doc	textual data, documents
word processor	wp, tex, rtf, doc	various word-processor formats
library	lib, a, so, dll	libraries of routines for programmers
print or view	ps, pdf, jpg	ASCII or binary file in a format for printing or viewing
archive	arc, zip, tar	related files grouped into one file, sometimes compressed, for archiving or storage
multimedia	mpeg, mov, rm, mp3, avi	binary file containing audio or A/V information

OPERATING SYSTEMS

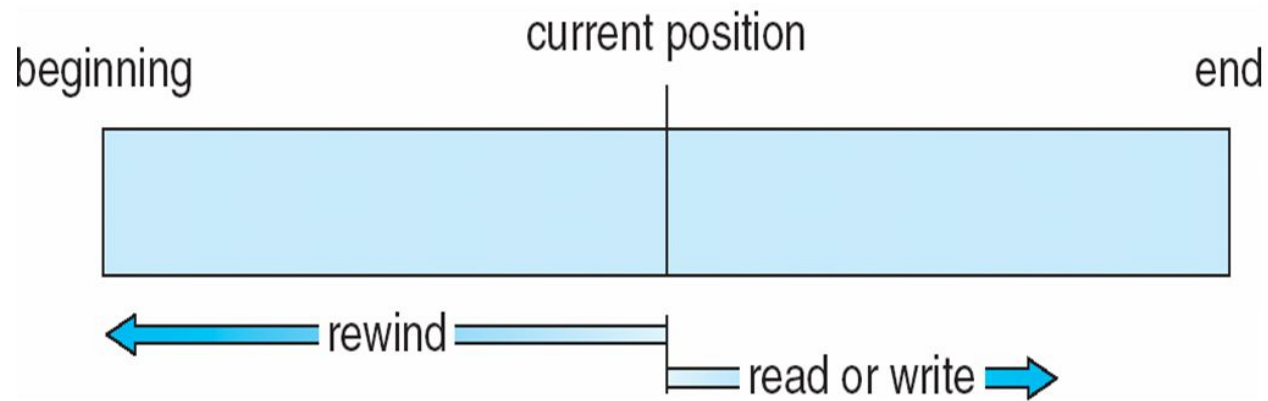
File Structure

- None - sequence of words, bytes
- Simple record structure
 - Lines
 - Fixed length
 - Variable length
- Complex Structures
 - Formatted document
 - Relocatable load file
- Can simulate last two with first method by inserting appropriate control characters
- Who decides:
 - Operating system
 - Program



OPERATING SYSTEMS

Access Methods



PES
UNIVERSITY

CELEBRATING 50 YEARS

- **Sequential Access**
 - read next
 - write next
 - reset
- **Direct Access (or relative access)** – file is fixed length **logical records**
 - read n
 - write n
 - position to n
 - read next
 - write next

n = **relative block number (i.e. index relative to the beginning of the file)**
- Relative block numbers allow OS to decide where file should be placed

sequential access	implementation for direct access
<i>reset</i>	<i>cp = 0;</i>
<i>read next</i>	<i>read cp;</i> <i>cp = cp + 1;</i>
<i>write next</i>	<i>write cp;</i> <i>cp = cp + 1;</i>

- *cp* = current position
- Some systems allow only sequential file access; others allow only direct access



THANK YOU

Pavan A C

Department of Computer Science & Engineering

pavanac@pes.edu

OPERATING SYSTEMS

UE24CS242B



File Management

Pavan A C

Department of Computer Science & Engineering

OPERATING SYSTEMS

Slides Credits for all the PPTs of this course

- The slides/diagrams in this course are an **adaptation, combination, and enhancement** of material from the following resources and persons:
1. Slides of Operating System Concepts, Abraham Silberschatz, Peter Baer Galvin, Greg Gagne - 9th edition 2013 and some slides from 10th edition 2018
 2. Some conceptual text and diagram from Operating Systems - Internals and Design Principles, William Stallings, 9th edition 2018
 3. Some presentation transcripts from A. Frank – P. Weisberg
 4. Some conceptual text from Operating Systems: Three Easy Pieces, Remzi Arpaci-Dusseau, Andrea Arpaci Dusseau



OPERATING SYSTEMS

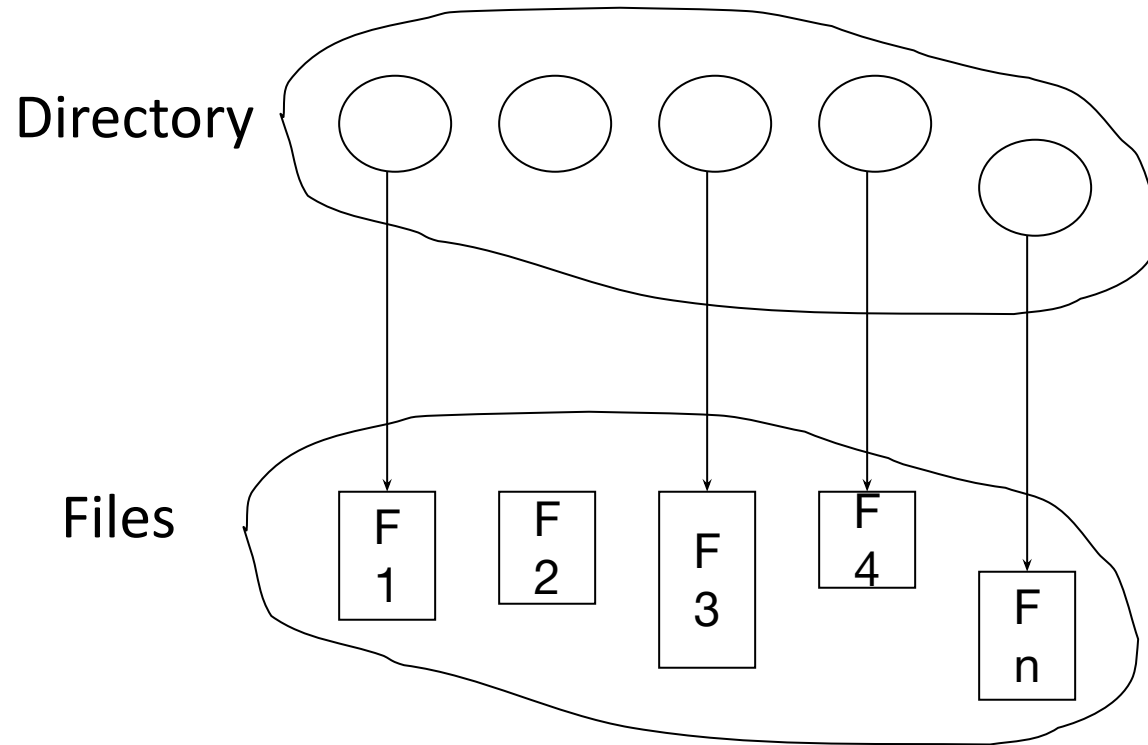


File System

Pavan A C

Department of Computer Science & Engineering

- A collection of nodes containing information about all files

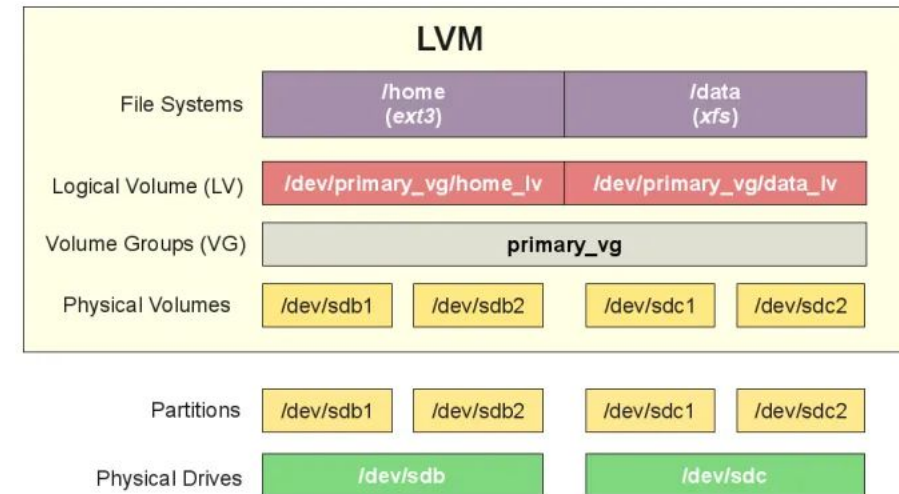


Both the directory structure and the files reside on disk

OPERATING SYSTEMS

Disk Structure

- Disk can be subdivided into **partitions**
- Disks or partitions can be **RAID** protected against failure
- Disk or partition can be used **raw** – without a file system, or **formatted** with a file system
- Partitions also known as minidisks, slices
- Entity containing file system known as a **volume**
- Each volume containing file system also tracks that file system's info in **device directory** or **volume table of contents**
- As well as **general-purpose file systems** there are many **special-purpose file systems**, frequently all within the same operating system or computer



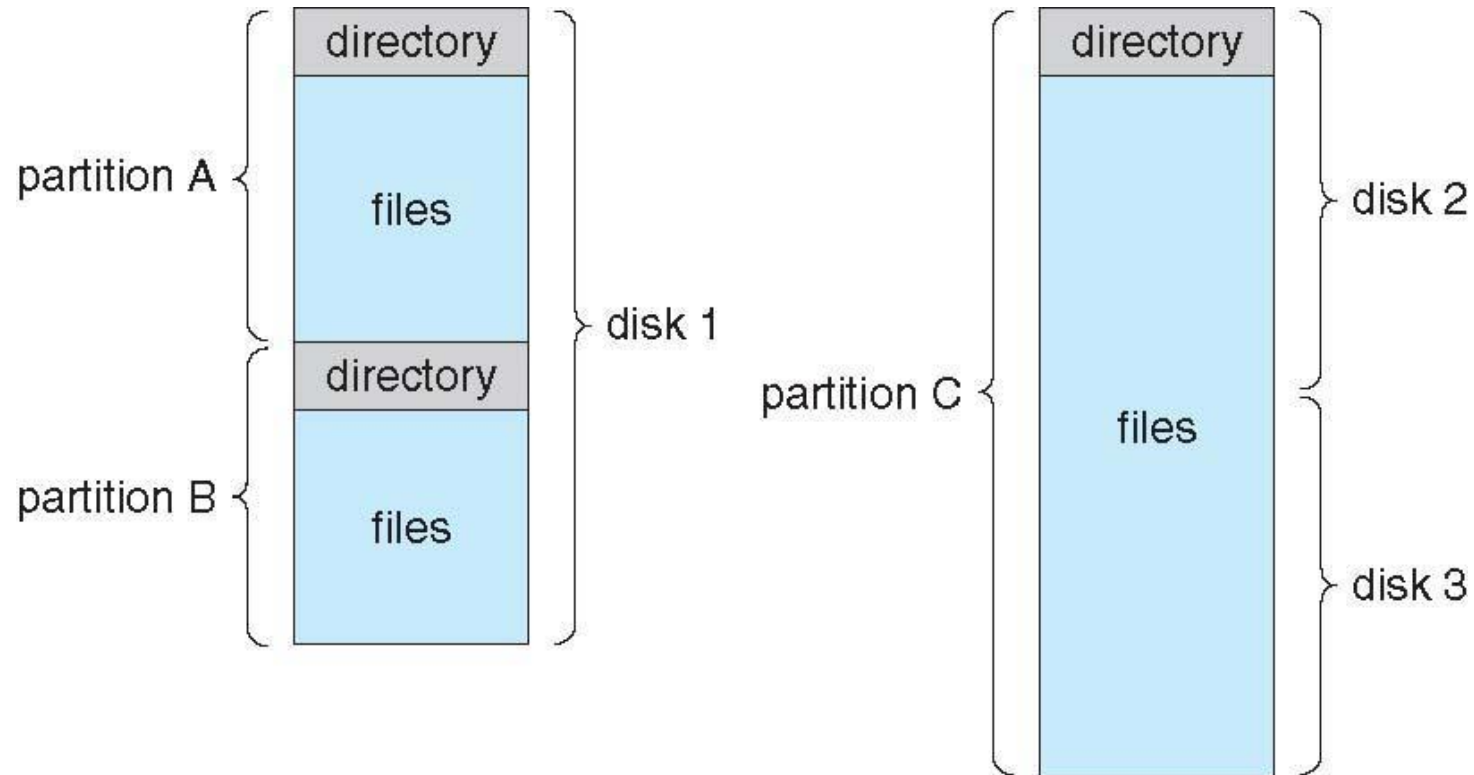
OPERATING SYSTEMS

A Typical File-system Organization



PES
UNIVERSITY

CELEBRATING 50 YEARS



OPERATING SYSTEMS

Operations Performed on Directory

- Search for a file
- Create a file
- Delete a file
- List a directory
- Rename a file
- Traverse the file system



OPERATING SYSTEMS

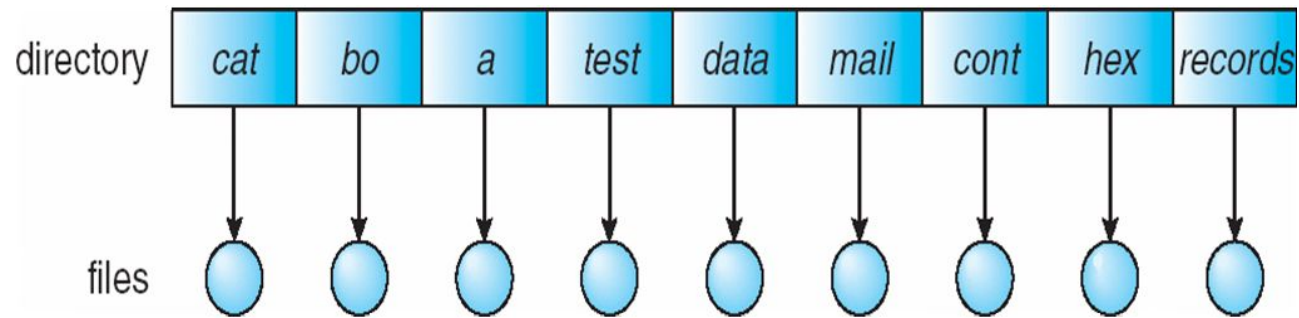
Directory Organization



The directory is organized logically to obtain

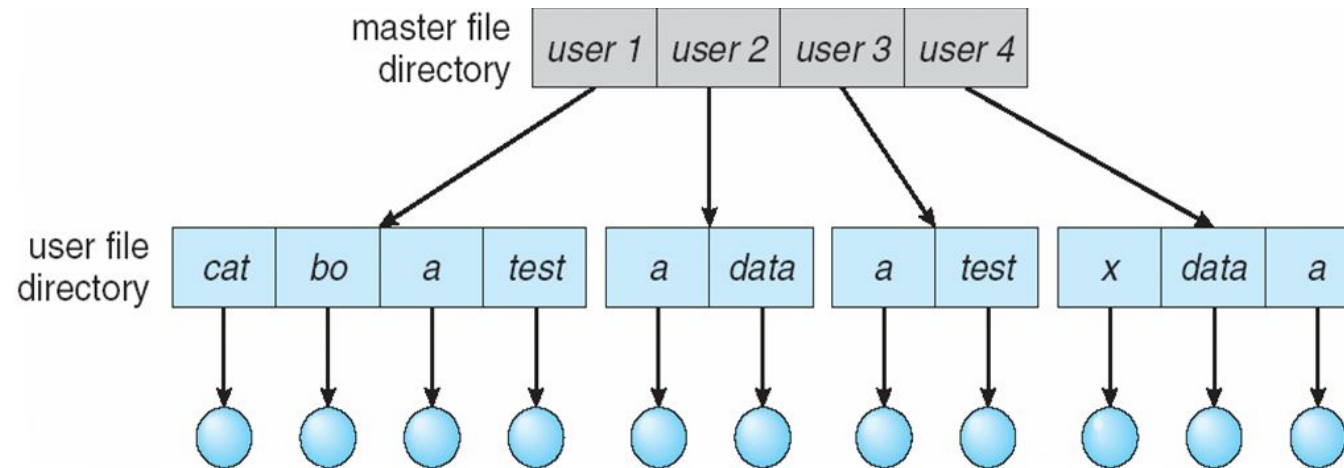
- **Efficiency** – locating a file quickly
- **Naming** – convenient to users
 - Two users can have same name for different files
 - The same file can have several different names
- **Grouping** – logical grouping of files by properties, (e.g., all Java programs, all games, ...)

- A single directory for all users

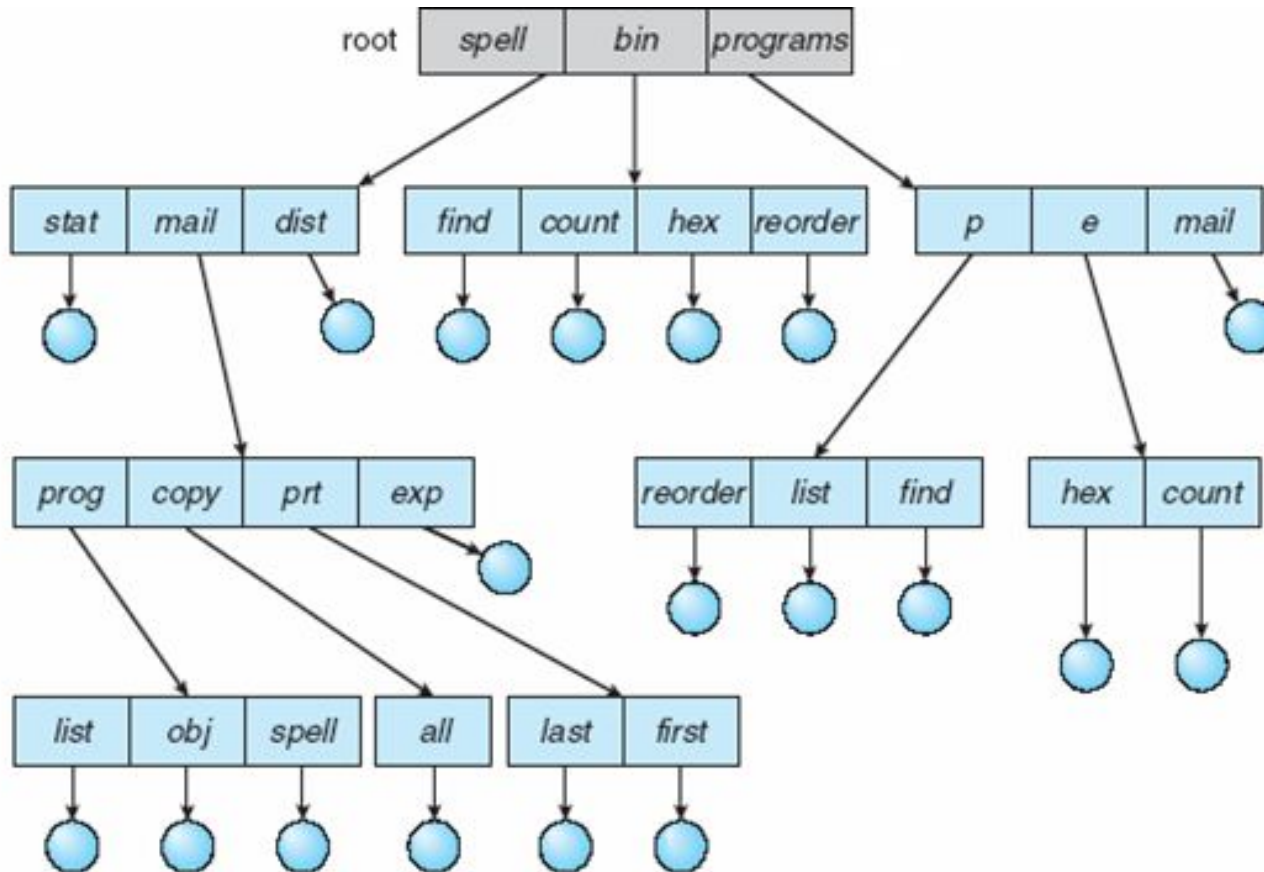


- Naming problem
- Grouping problem

- Separate directory for each user

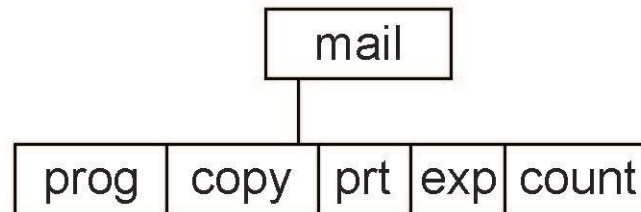


- Path name
- Can have the same file name for different user
- Efficient searching
- No grouping capability



- A tree structure is the most common directory structure.
- The tree has a root directory, and every file in the system have a unique path.

- Efficient searching
- Grouping Capability
- Current directory (working directory)
 - **cd /spell/mail/prog**
 - **type list**
- We cannot share files



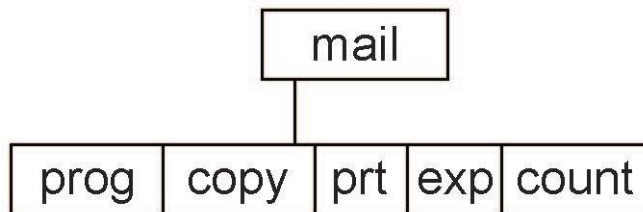
Deleting “mail” $\Rightarrow$ deleting the entire subtree rooted by “mail”

OPERATING SYSTEMS

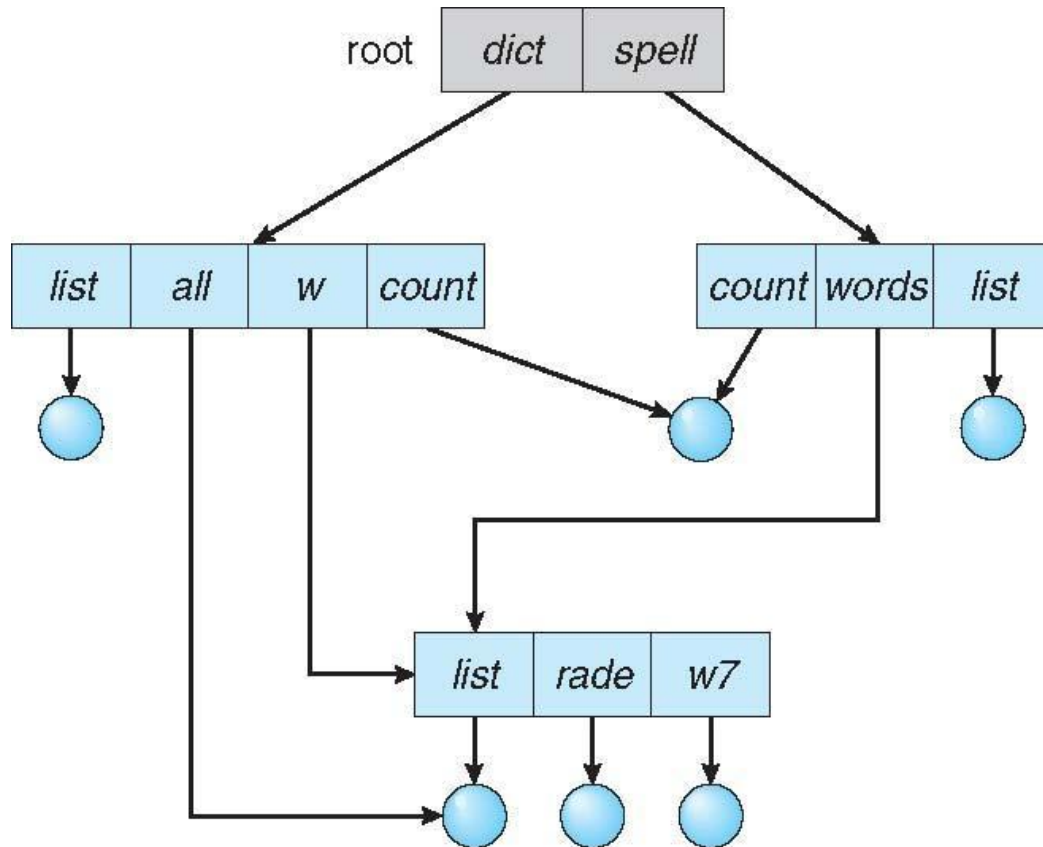
Tree-Structured Directories (Cont.)

- **Absolute** or **relative** path name
- Creating a new file is done in current directory
- Delete a file
rm <file-name>
- Creating a new subdirectory is done in current directory
mkdir <dir-name>

- ❑ Example: if in current directory **/mail**
mkdir count



Deleting “mail” $\Rightarrow$ deleting the entire subtree rooted by “mail”



- An acyclic graph is a graph with no cycle and allows to share subdirectories and files.
- The same file or subdirectories may be in two different directories
- It is used in a situation like when two programmers are working on a joint project and they need to access files.

- Two different names (aliasing)
- If *dict* deletes *list* $\Rightarrow$ dangling pointer

Solutions:

- Backpointers, so we can delete all pointers
Variable size records a problem
- Backpointers using a daisy chain organization
- Entry-hold-count solution
- New directory entry type
 - **Link** – another name (pointer) to an existing file
 - **Resolve the link** – follow pointer to locate the file

OPERATING SYSTEMS

General Graph Directory

Advantages:

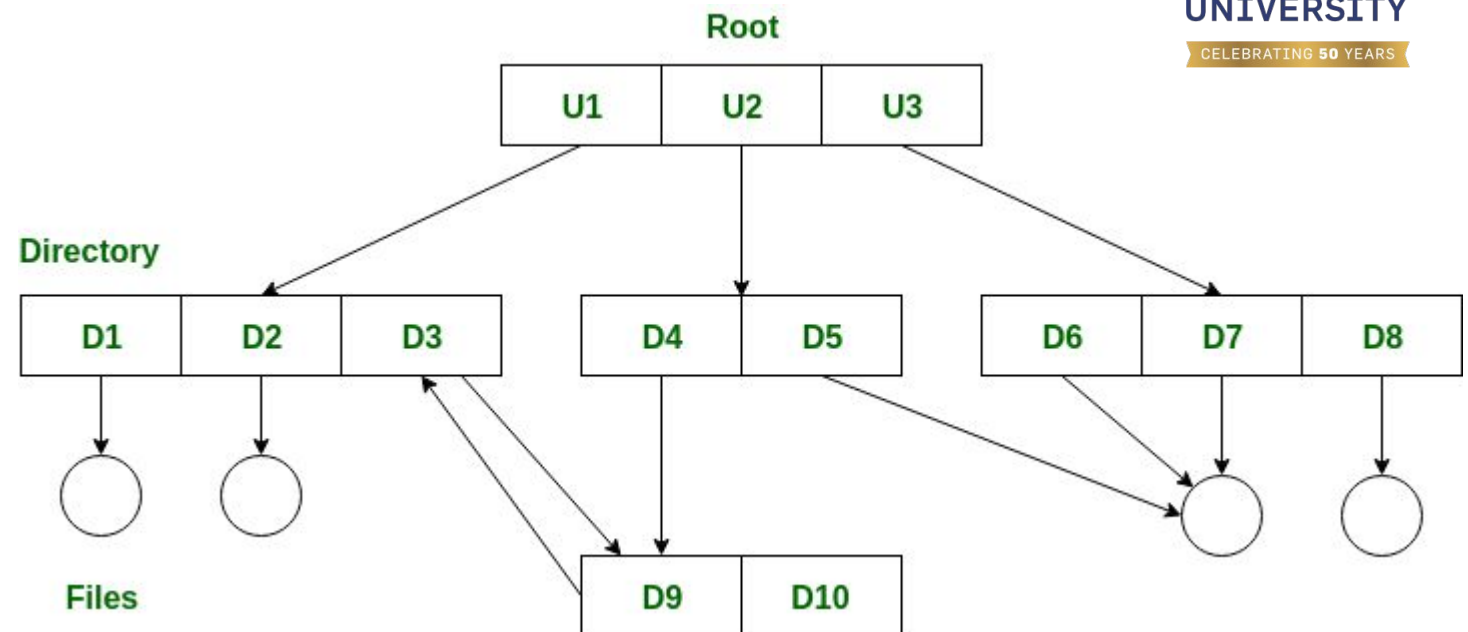
It allows cycles within a dir structure.

Multiple directories can be derived from more than one parent dir.

Disadvantages:

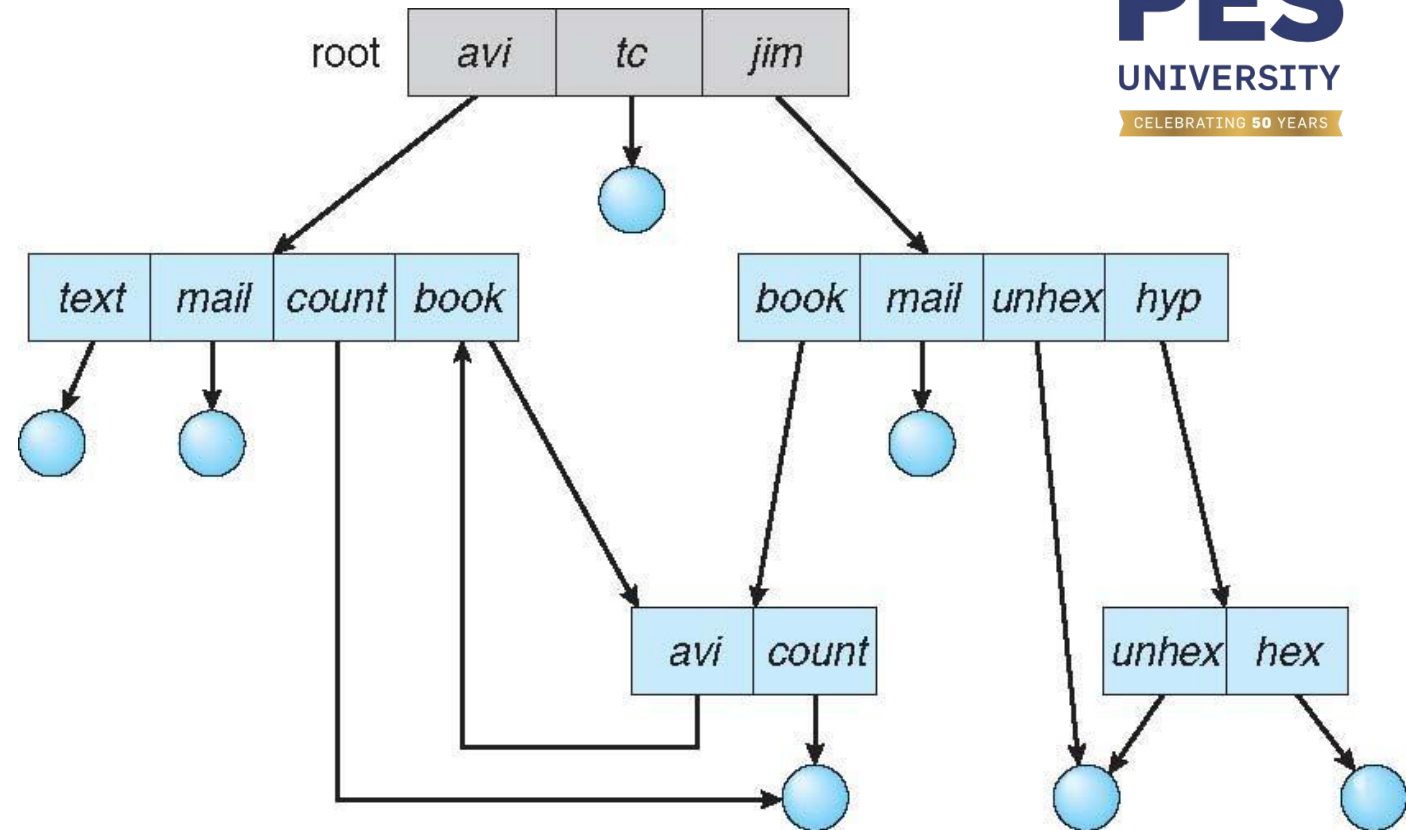
It is more costly than others.

It needs garbage collection (traversing the entire file system, marking everything that can be accessed)





- How do we guarantee no cycles?
 - Allow only links to file not subdirectories
 - **Garbage collection**
 - Every time a new link is added use a cycle detection algorithm to determine whether it is OK





THANK YOU

Pavan A C

Department of Computer Science & Engineering

pavanac@pes.edu

OPERATING SYSTEMS

UE24CS242B



File Management

Pavan A C

Department of Computer Science & Engineering

OPERATING SYSTEMS

File System – File-System Mounting, File Sharing and File Protection

Pavan A C

Department of Computer Science & Engineering

OPERATING SYSTEMS

Slides Credits for all the PPTs of this course

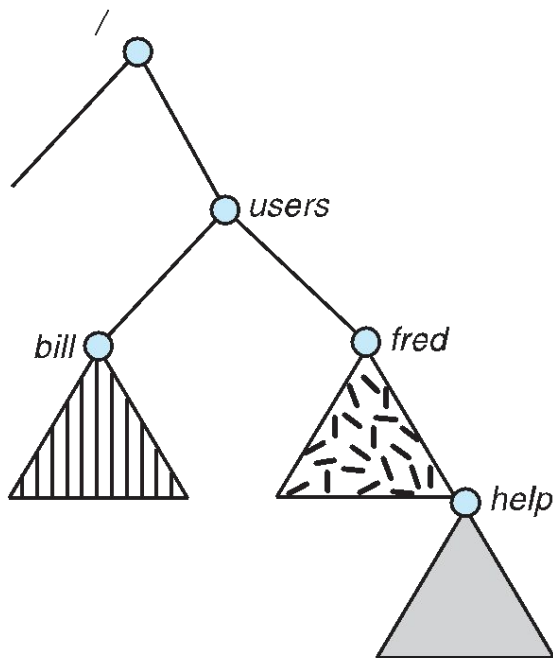
- The slides/diagrams in this course are an **adaptation, combination, and enhancement** of material from the following resources and persons:
1. Slides of Operating System Concepts, Abraham Silberschatz, Peter Baer Galvin, Greg Gagne - 9th edition 2013 and some slides from 10th edition 2018
 2. Some conceptual text and diagram from Operating Systems - Internals and Design Principles, William Stallings, 9th edition 2018
 3. Some presentation transcripts from A. Frank – P. Weisberg
 4. Some conceptual text from Operating Systems: Three Easy Pieces, Remzi Arpaci-Dusseau, Andrea Arpaci Dusseau



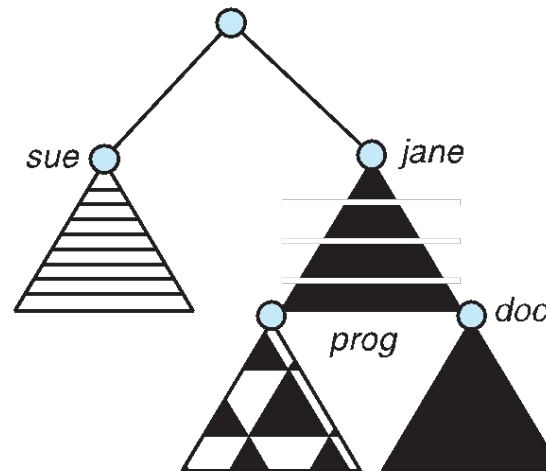
OPERATING SYSTEMS

File System Mounting

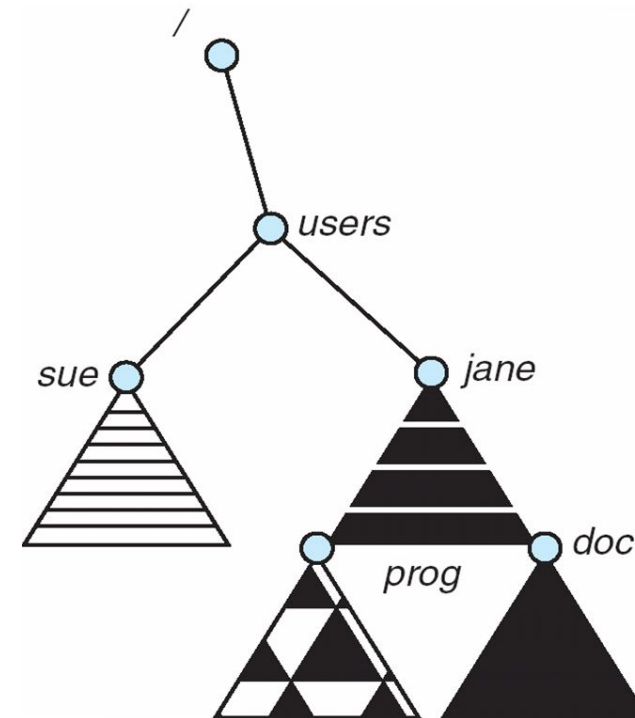
- A file system must be **mounted** before it can be accessed
- Fig (a) shows Existing File System
- A unmounted file system (i.e., Fig. (b)) is mounted at a **mount point**
- Fig (c) shows the effect of mounting



(a)



(b)



(c)

OPERATING SYSTEMS

File Sharing

- Sharing of files on multi-user systems is desirable
- Sharing may be done through a **protection** scheme
- On distributed systems, files may be shared across a network
- Network File System (NFS) is a common distributed file-sharing method
- If multi-user system
 - **User IDs** identify users, allowing permissions and protections to be per-user
 - **Group IDs** allow users to be in groups, permitting group access rights
 - Owner of a file / directory
 - Group of a file / directory



OPERATING SYSTEMS

File Sharing – Remote File Systems

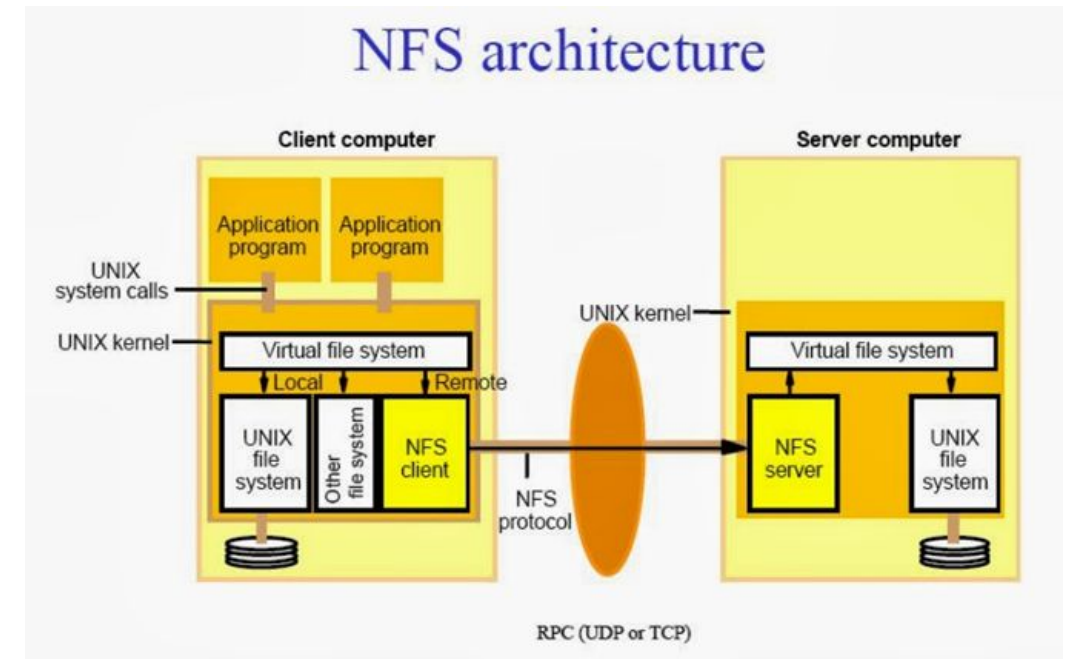
- Uses networking to allow file system access between systems
 - Manually via programs like FTP
 - Automatically, seamlessly using **distributed file systems**
 - Semi automatically via the **world wide web**



OPERATING SYSTEMS

File Sharing – Remote File Systems (Cont.)

- **Client-server** model allows clients to mount remote file systems from servers
 - Server can serve multiple clients
 - Client and user-on-client identification is insecure or complicated
 - **NFS** is standard UNIX client-server file sharing protocol
 - **CIFS** is standard Windows protocol
 - Standard operating system file calls are translated into remote calls
- Distributed Information Systems (**distributed naming services**) such as LDAP, DNS, NIS, Active Directory implement unified access to information needed for remote computing



- File owner/creator should be able to control:
 - what can be done
 - by whom
- Types of access
 - **Read**
 - **Write**
 - **Execute**
 - **Append**
 - **Delete**
 - **List**

OPERATING SYSTEMS

Access Lists and Groups



- Mode of access: read, write, execute
- Three classes of users on Unix / Linux

RWX

a) **owner access** 7 $\Rightarrow$ 1 1 1

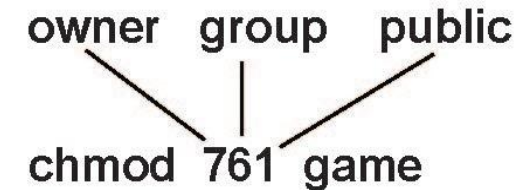
RWX

b) **group access** 6 $\Rightarrow$ 1 1 0

RWX

c) **public access** 1 $\Rightarrow$ 0 0 1

- Ask manager to create a group (unique name), say G, and add some users to the group.
- For a particular file (say *game*) or subdirectory, define an appropriate access.



Attach a group to a file
chgrp G game



THANK YOU

Pavan A C

Department of Computer Science and Engineering

pavanac@pes.edu

OPERATING SYSTEMS

UE24CS242B



File Management

Pavan A C

Department of Computer Science & Engineering

OPERATING SYSTEMS



Implementing File-Systems

Pavan A C

Department of Computer Science & Engineering

OPERATING SYSTEMS

Slides Credits for all the PPTs of this course

- The slides/diagrams in this course are an **adaptation, combination,** and **enhancement** of material from the following resources and persons:

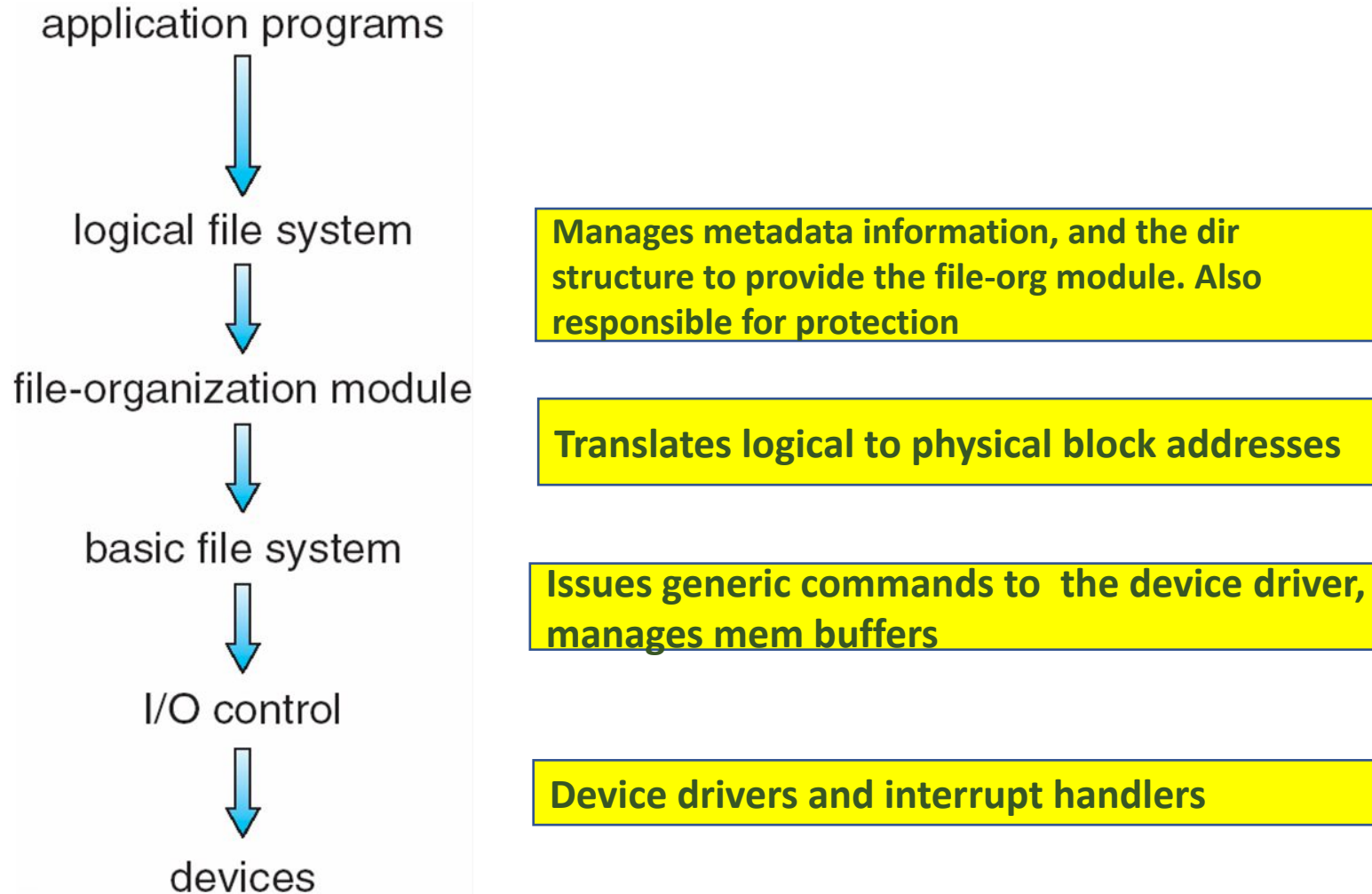
1. Slides of Operating System Concepts, Abraham Silberschatz, Peter Baer Galvin, Greg Gagne - 9th edition 2013 and some slides from 10th edition 2018
2. Some conceptual text and diagram from Operating Systems - Internals and Design Principles, William Stallings, 9th edition 2018
3. Some presentation transcripts from A. Frank – P. Weisberg
4. Some conceptual text from Operating Systems: Three Easy Pieces, Remzi Arpaci-Dusseau, Andrea Arpaci Dusseau



OPERATING SYSTEMS

Layered File System

The file system is generally composed of many different levels



OPERATING SYSTEMS

File-System Structure



- File structure
 - Logical storage unit
 - Collection of related information
- **File system** resides on secondary storage (disks)
 - Provided user interface to storage, mapping logical to physical
 - Provides efficient and convenient access to disk by allowing data to be stored, located retrieved easily
- Disk provides in-place rewrite and random access
 - I/O transfers performed in **blocks** of **sectors** (usually 512 bytes)
- **File control block (FCB)** – storage structure consisting of information about a file including ownership, permissions, and location of the file contents
- **Device driver** controls the physical device
- File system organized into layers

- **Device drivers** manage I/O devices at the I/O control layer
 - Given commands like “read drive1, cylinder 72, track 2, sector 10, into memory location 1060” outputs low-level hardware specific commands to hardware controller
- **Basic file system** given command like “retrieve block 123” translates to device driver
- Also manages memory buffers and caches (allocation, freeing, replacement)
 - Buffers hold data in transit
 - Caches hold frequently used data

OPERATING SYSTEMS

File System Layers (Cont.)

- **File organization module** understands files, logical address, and physical blocks
- Translates logical block # to physical block #
- Manages free space, disk allocation
- **Logical file system** manages metadata information
 - Translates file name into file number, file handle, location by maintaining file control blocks (**inodes** in UNIX)
 - Directory management
 - Protection



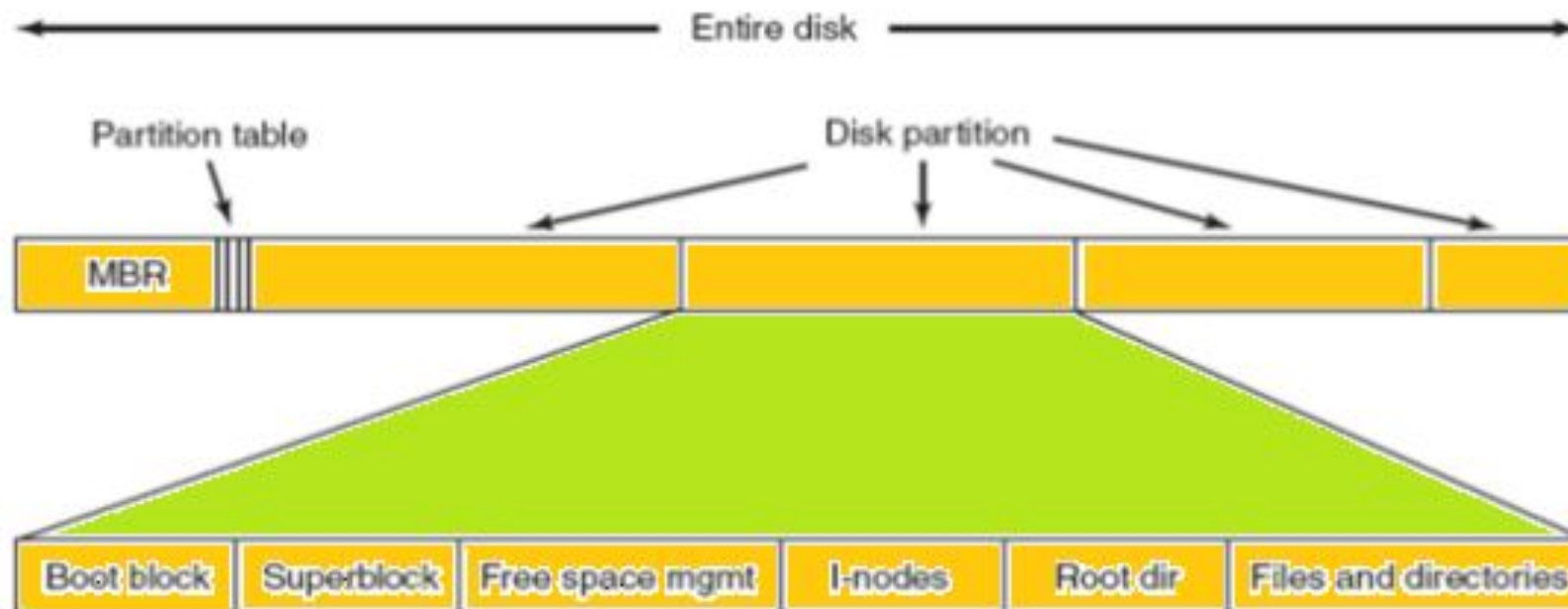
OPERATING SYSTEMS

File-System Implementation



PES
UNIVERSITY

CELEBRATING 50 YEARS



A possible file system layout.

- We have system calls at the API level, but how do we implement their functions?
 - On-disk and in-memory structures
- **Boot control block** contains info needed by system to boot OS from that volume
 - Needed if volume contains OS, usually first block of volume
- **Volume control block (superblock, master file table)** contains volume details
 - Total # of blocks, # of free blocks, block size, free block pointers or array
- Directory structure organizes the files
 - Names and inode numbers, master file table

- Per-file **File Control Block (FCB)** contains many details about the file
 - inode number, permissions, size, dates
 - NFTS stores into in master file table using relational DB structures

file permissions
file dates (create, access, write)
file owner, group, ACL
file size
file data blocks or pointers to file data blocks

OPERATING SYSTEMS

In-Memory File System Structures

- Mount table storing file system mounts, mount points, file system types
- The following figure illustrates the necessary file system structures provided by the operating systems
- Figure (a) refers to opening a file
- Figure (b) refers to reading a file
- Plus buffers hold data blocks from secondary storage
- Open returns a file handle for subsequent use
- Data from read eventually copied to specified user process memory address



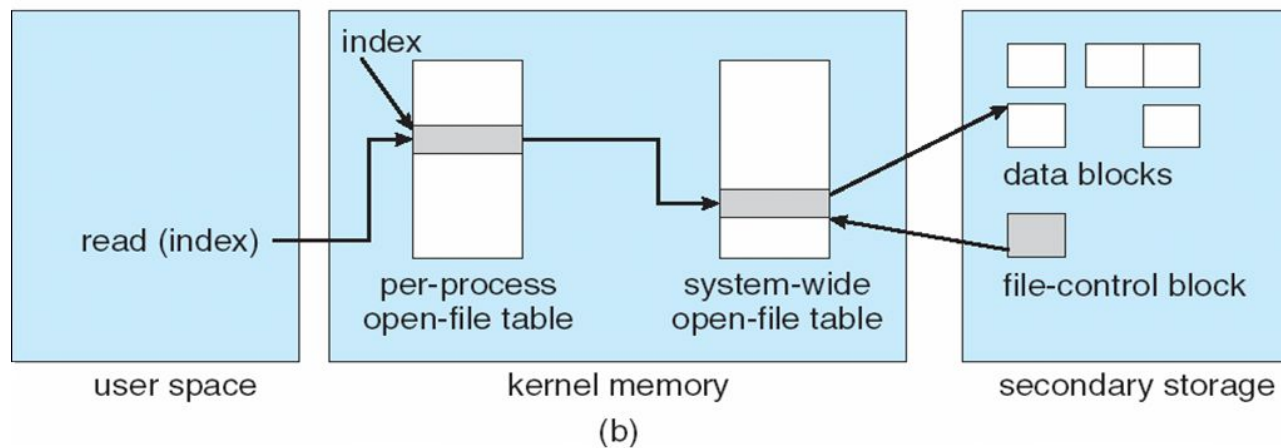
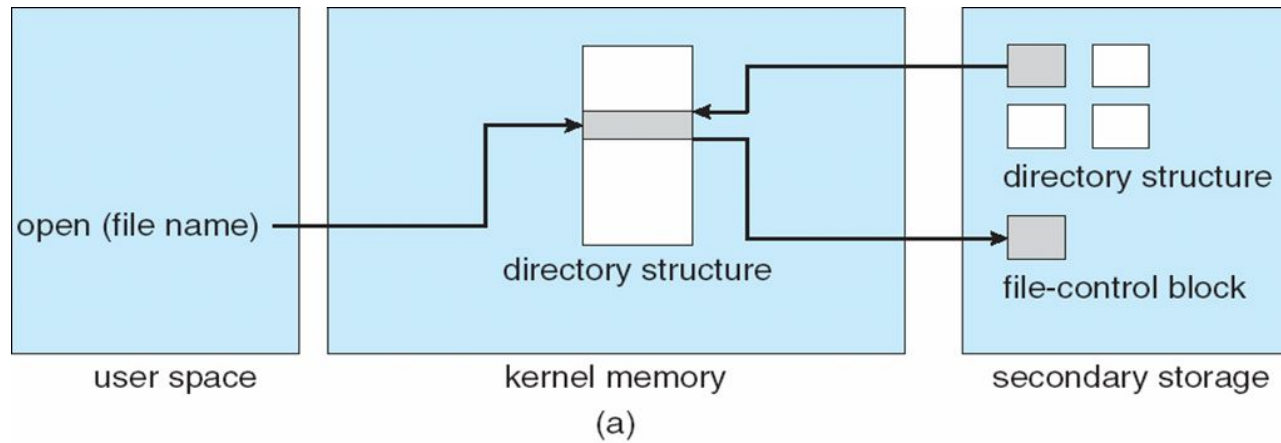
OPERATING SYSTEMS

In-Memory File System Structures



PES
UNIVERSITY

CELEBRATING 50 YEARS



OPERATING SYSTEMS

Partitions and Mounting

- Partition can be a volume containing a file system (“cooked”) or **raw** – just a sequence of blocks with no file system
- Boot block can point to boot volume or boot loader set of blocks that contain enough code to know how to load the kernel from the file system
 - Or a boot management program for multi-os booting
- **Root partition** contains the OS, other partitions can hold other OS’s, other file systems, or be raw
 - Mounted at boot time
 - Other partitions can mount automatically or manually
- At mount time, file system consistency checked
 - Is all metadata correct?
 - If not, fix it, try again
 - If yes, add to mount table, allow access



OPERATING SYSTEMS

Virtual File Systems

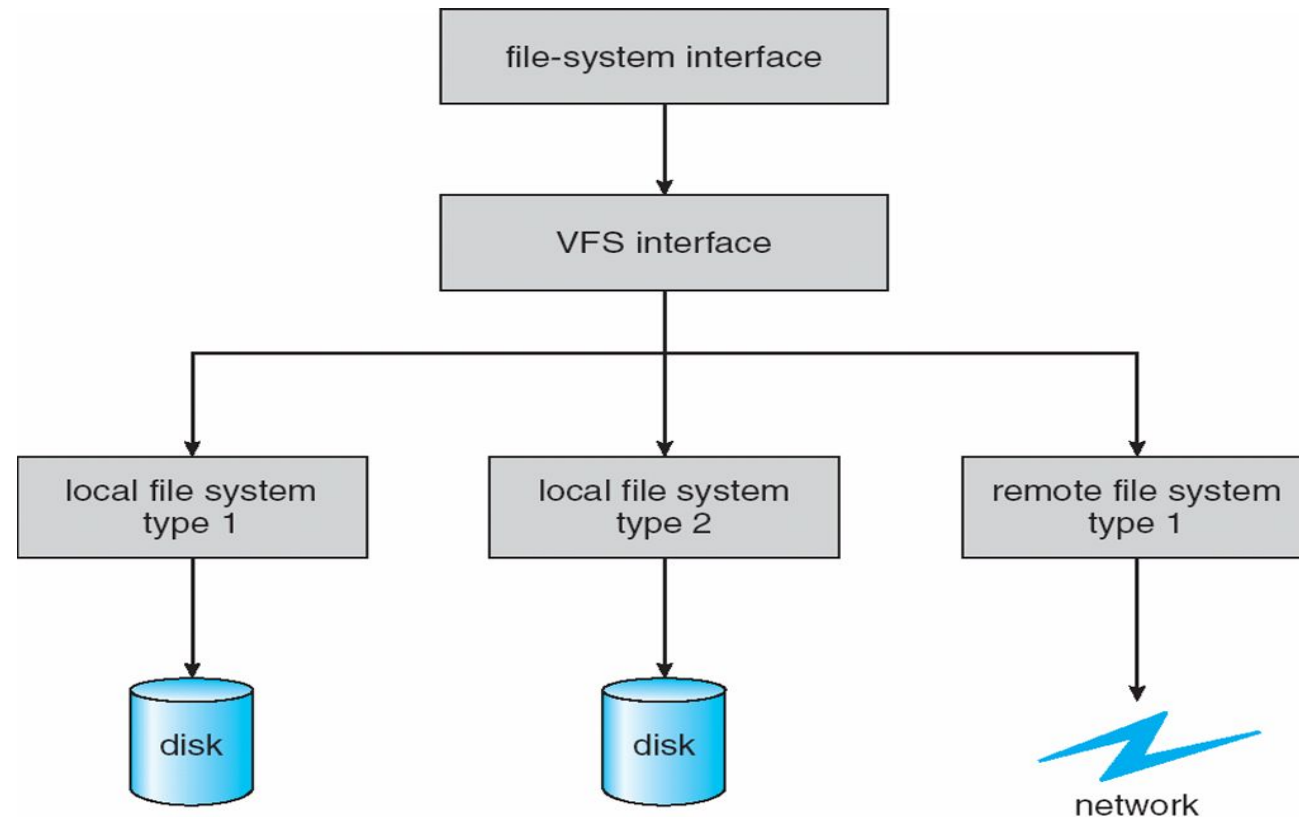
- **Virtual File Systems (VFS)** on Unix provide an object-oriented way of implementing file systems
- VFS allows the same system call interface (the API) to be used for different types of file systems
 - Separates file-system generic operations from implementation details
 - Implementation can be one of many file systems types, or network file system
 - 4 Implements **vnodes** which hold inodes or network file details
 - Then dispatches operation to appropriate file system implementation routines



OPERATING SYSTEMS

Virtual File Systems

- The API is to the VFS interface, rather than any specific type of file system (FS)
- VFS provides a single set of commands for the kernel and developers to access all types of FSs
- VFS software calls the specific device driver required to interface to various types of FSs
- Device driver interprets the standard set of FS commands to a specific type of FS on the partition or logical volume



- For example, Linux has four object types:
 - **Inode object, file object, superblock object, dentry object**
- VFS defines set of operations on the objects that must be implemented
 - Every object has a pointer to a function table
Function table has addresses of routines to implement that function on that object
For example:
 - **int open(. . .)**—Open a file
 - **int close(. . .)**—Close an already-open file
 - **ssize_t read(. . .)**—Read from a file
 - **ssize_t write(. . .)**—Write to a file
 - **int mmap(. . .)**—Memory-map a file



THANK YOU

Pavan A C

Department of Computer Science and Engineering

pavanac@pes.edu

OPERATING SYSTEMS

UE24CS242B



File Management

Pavan A C

Department of Computer Science & Engineering

OPERATING SYSTEMS



Implementing File-Systems – Disk Space Allocation Methods

Pavan A C

Department of Computer Science & Engineering

OPERATING SYSTEMS

Slides Credits for all the PPTs of this course

- The slides/diagrams in this course are an **adaptation, combination, and enhancement** of material from the following resources and persons:
1. Slides of Operating System Concepts, Abraham Silberschatz, Peter Baer Galvin, Greg Gagne - 9th edition 2013 and some slides from 10th edition 2018
 2. Some conceptual text and diagram from Operating Systems - Internals and Design Principles, William Stallings, 9th edition 2018
 3. Some presentation transcripts from A. Frank – P. Weisberg
 4. Some conceptual text from Operating Systems: Three Easy Pieces, Remzi Arpaci-Dusseau, Andrea Arpaci Dusseau



OPERATING SYSTEMS

Allocation Methods - Contiguous

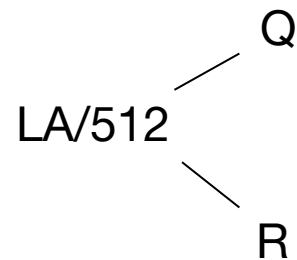
- An allocation method refers to how disk blocks are allocated for files:
- **Contiguous allocation** – each file occupies set of contiguous blocks
 - Best performance in most cases
 - Simple – only starting location (block #) and length (number of blocks) are required
 - Supports both sequential and direct access
 - Problems include finding space for file, knowing file size, external fragmentation, need for **compaction off-line (downtime)** or **on-line**



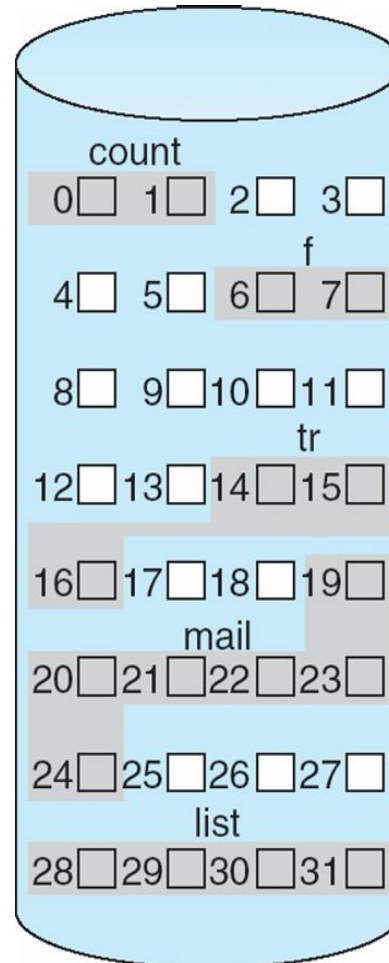
OPERATING SYSTEMS

Contiguous Allocation

- Mapping from logical to physical



- LA – Length of the area allocated for this file
- Q = displacement into index table
- Block to be accessed = Q + starting address
- Displacement into block = R



directory

file	start	length
count	0	2
tr	14	3
mail	19	6
list	28	4
f	6	2

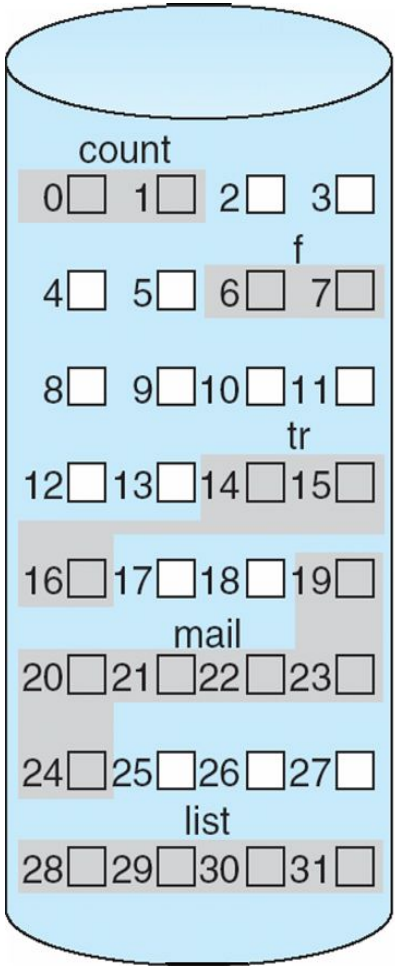
OPERATING SYSTEMS

Contiguous Allocation



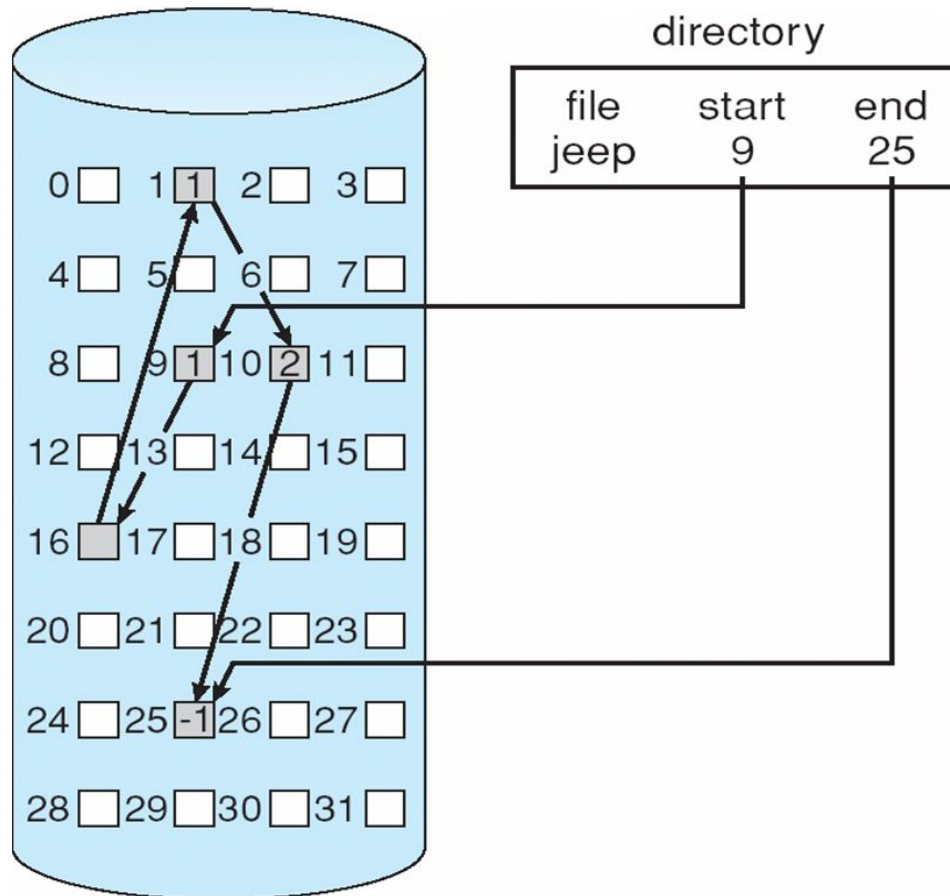
PES
UNIVERSITY

CELEBRATING 50 YEARS



directory

file	start	length
count	0	2
tr	14	3
mail	19	6
list	28	4
f	6	2



- Each block contains a pointer to the next block.
- If each block is 512 bytes and a disk address (pointer) requires 4 bytes then the user sees blocks of 508 bytes (i.e. some disk space is wasted)
- Collect blocks into multiples called **clusters** and allocate clusters rather than blocks

- **Linked allocation** – each file a linked list of blocks
 - File ends with null pointer
 - No external fragmentation
 - Each block contains pointer to next block
 - No compaction, external fragmentation
 - Free space management system called when new block needed
 - Improve efficiency by clustering blocks into groups but increases internal fragmentation
 - Reliability can be a problem
 - Locating a block can take many I/Os and disk seeks

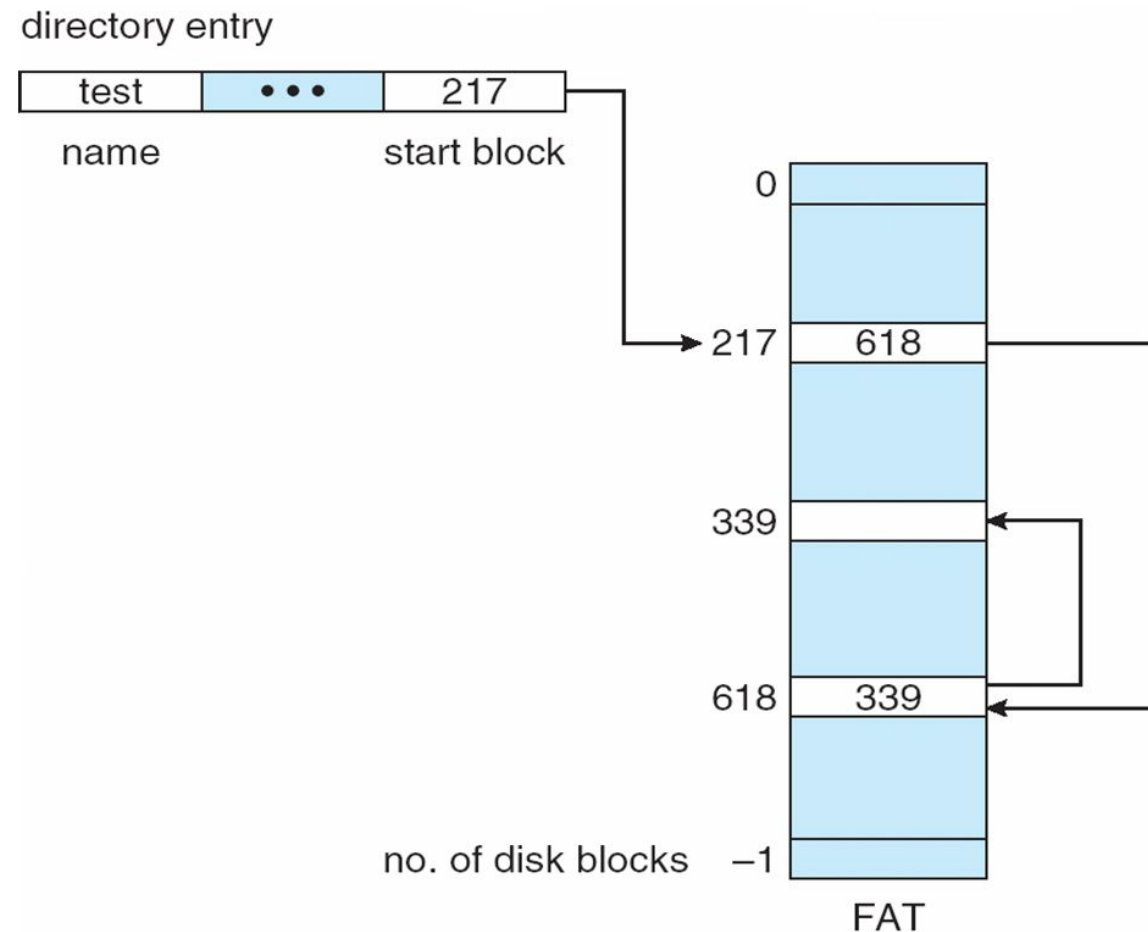
OPERATING SYSTEMS

Allocation Methods – Linked (Cont.)

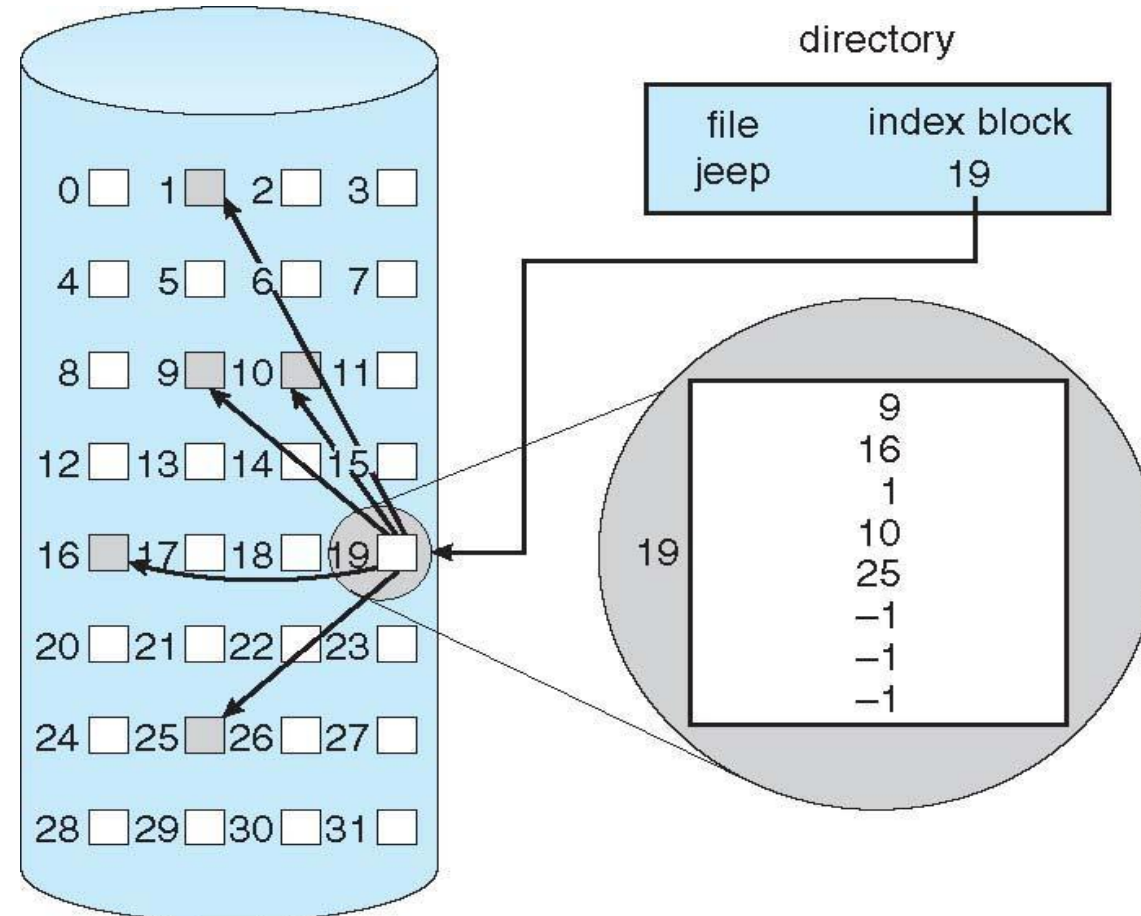
- FAT (File Allocation Table) variation
 - Beginning of volume has table, indexed by block number
 - Much like a linked list, but faster on disk and cacheable
 - New block allocation simple



- Unused block is indicated by a table value of 0
- To allocate a new block to a file, find the first 0-valued table entry and replace the previous EOF value with the address of the new block.
- Then replace 0 with EOF value



- In this scheme, a special block known as the **Index block** contains the pointers to all the blocks occupied by a file.
- Each file has its own index block. The *i*th entry in the index block contains the disk address of the *i*th file block.
- The directory entry contains the address of the index block as shown in the figure.



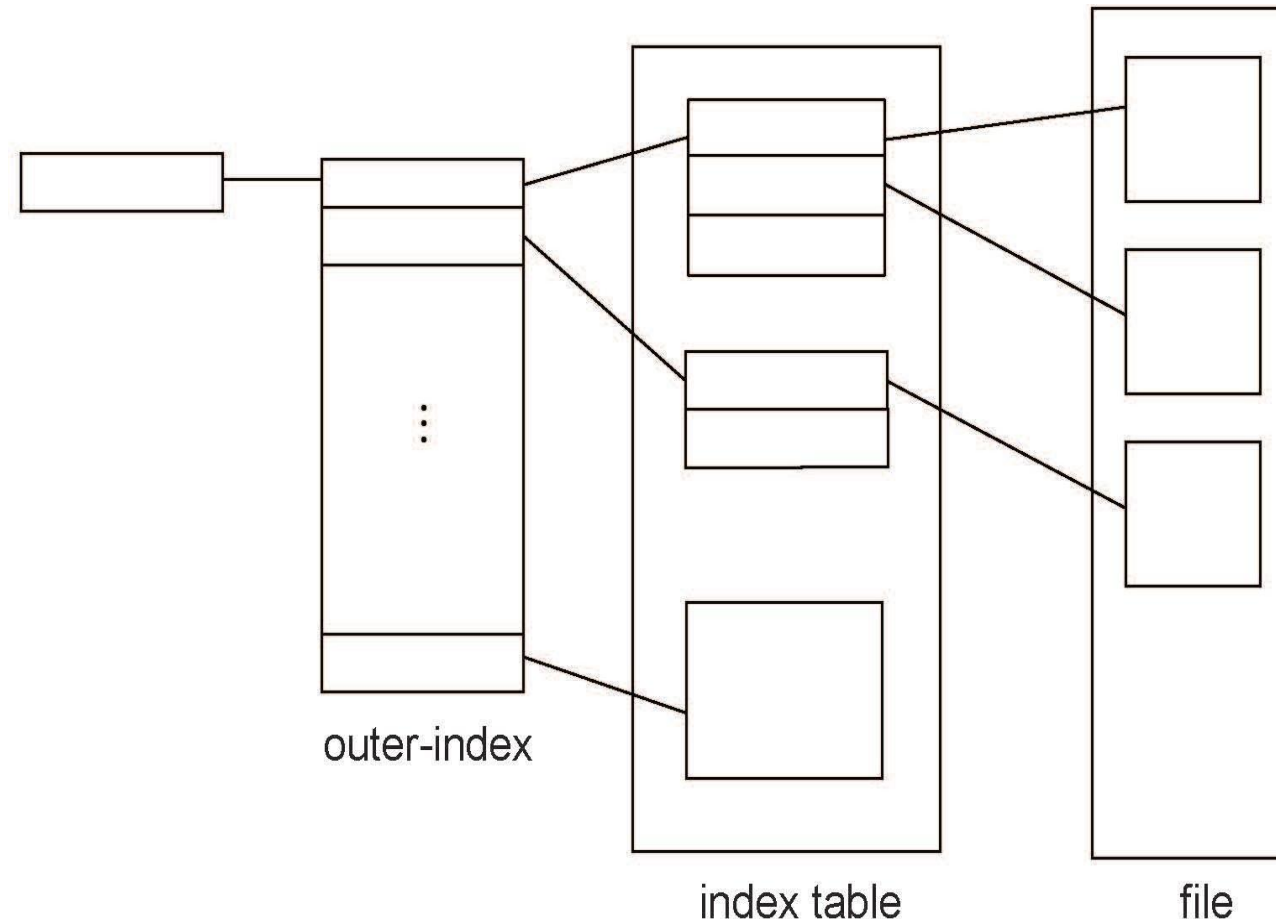
Advantages:

- This supports direct access to the blocks occupied by the file and therefore provides fast access to the file blocks.
- It overcomes the problem of external fragmentation.

Disadvantages:

- The pointer overhead for indexed allocation is greater than linked allocation.
- For very small files, say files that expand only 2-3 blocks, the indexed allocation would keep one entire block (index block) for the pointers which is inefficient in terms of memory utilization. (Note: In linked allocation we lose the space of only 1 pointer per block.)

- For files that are very large, single index block may not be able to hold all the pointers.
- Other Schemes such as Linked scheme, Multilevel index and Combined Scheme are used.





THANK YOU

Pavan A C

Department of Computer Science and Engineering

pavanac@pes.edu

OPERATING SYSTEMS

UE24CS242B



File Management

Pavan A C

Department of Computer Science & Engineering

OPERATING SYSTEMS

File Management – Free space management, Efficiency and Performance

Pavan A C

Department of Computer Science & Engineering

OPERATING SYSTEMS

Slides Credits for all the PPTs of this course

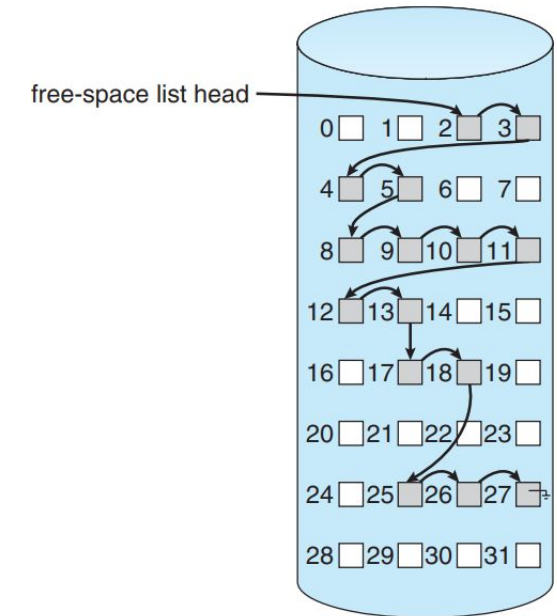
- The slides/diagrams in this course are an **adaptation, combination, and enhancement** of material from the following resources and persons:
1. Slides of Operating System Concepts, Abraham Silberschatz, Peter Baer Galvin, Greg Gagne - 9th edition 2013 and some slides from 10th edition 2018
 2. Some conceptual text and diagram from Operating Systems - Internals and Design Principles, William Stallings, 9th edition 2018
 3. Some presentation transcripts from A. Frank – P. Weisberg
 4. Some conceptual text from Operating Systems: Three Easy Pieces, Remzi Arpaci-Dusseau, Andrea Arpaci Dusseau

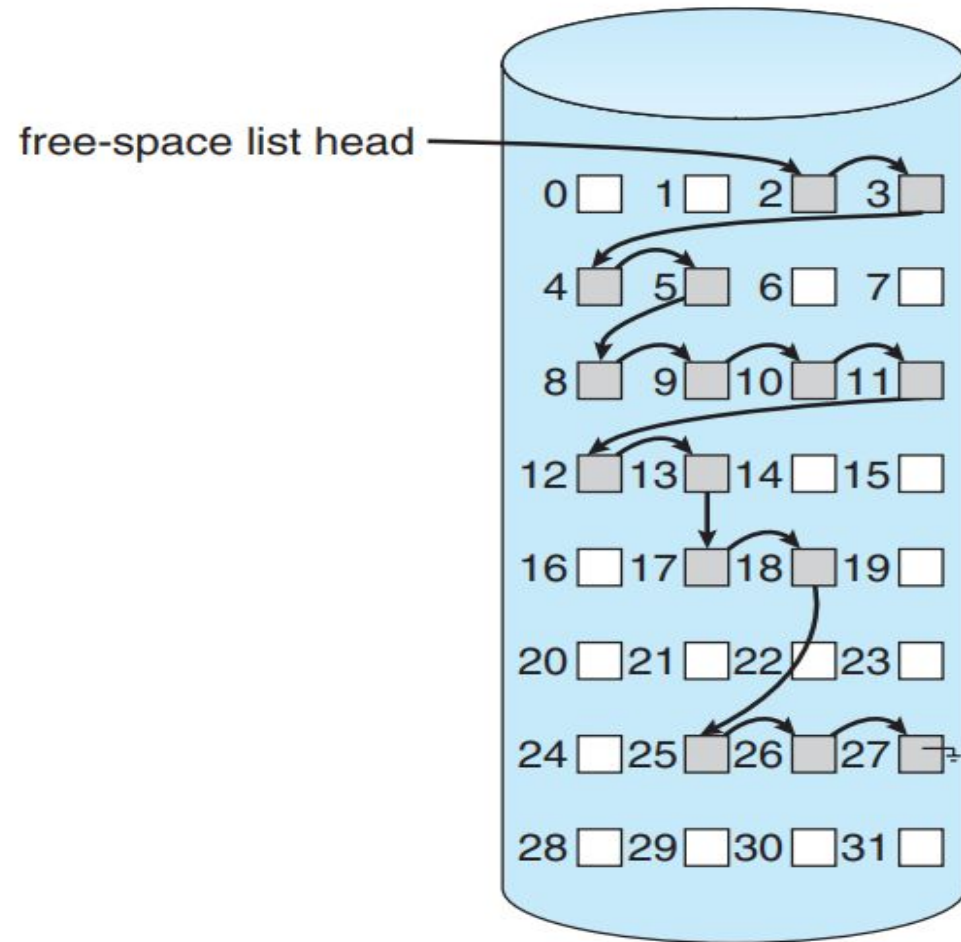


- Need to reuse the disk space from deleted files for new files
- To keep track of free disk space, the system maintains a **free-space list**
- The free-space list records all **free disk blocks**
- When a file is created, the free-space list is searched for the required amount of space and allocate that space to the new file. This space is then removed from the free-space list.
- When a file is deleted, its disk space is added to the free-space list

- The free-space list is implemented as a **bit map** or **bit vector**.
- Each block is represented by 1 bit.
- If the block is free, the bit is 1; if the block is allocated, the bit is 0
- For example, consider a disk where blocks 2, 3, 4, 5, 8, 9, 10, 11, 12, 13, 17, 18, 25, 26, and 27 are free and the rest of the blocks are allocated.
- The free-space bit map would be 001111001111110001100000011100000 ...
- The main advantage of this approach is its relative simplicity and its efficiency in finding the first free block or n consecutive free blocks on the disk

- Link together all the free disk blocks, keeping a pointer to the first free block in a special location on the disk and caching it in memory.
- The first free block contains a pointer to the next free disk block, and so on
- **Example:** Consider blocks 2, 3, 4, 5, 8, 9, 10, 11, 12, 13, 17, 18, 25, 26, and 27 are free and the rest of the blocks are allocated.
- The pointer would point to block 2 as the first free block. Block 2 would contain a pointer to block 3, which would point to block 4 and so on
- This scheme is not efficient; to traverse the list, one must read each block, which requires substantial I/O time.





- A modification of the free-list approach stores the addresses of n free blocks in the first free block.
- The first $n-1$ of these blocks are actually free.
- The last block contains the addresses of another n free blocks, and so on.
- The addresses of a large number of free blocks can now be found quickly, unlike the situation when the standard linked-list approach is used.

- Generally, several contiguous blocks may be allocated or freed simultaneously, particularly when space is allocated with the contiguous-allocation algorithm or through clustering.
- Rather than keeping a list of n free disk addresses, we can keep the address of the first free block and the number (n) of free contiguous blocks that follow the first block
- Each entry in the free-space list then consists of a disk address and a count.
- The entries can be stored in a balanced tree, rather than a linked list, for efficient lookup, insertion, and deletion.

Efficiency dependent on:

- Disk allocation and directory algorithms
- Types of data kept in file's directory entry
- Pre-allocation or as-needed allocation of metadata structures
- Fixed-size or varying-size data structures

Performance dependent on:

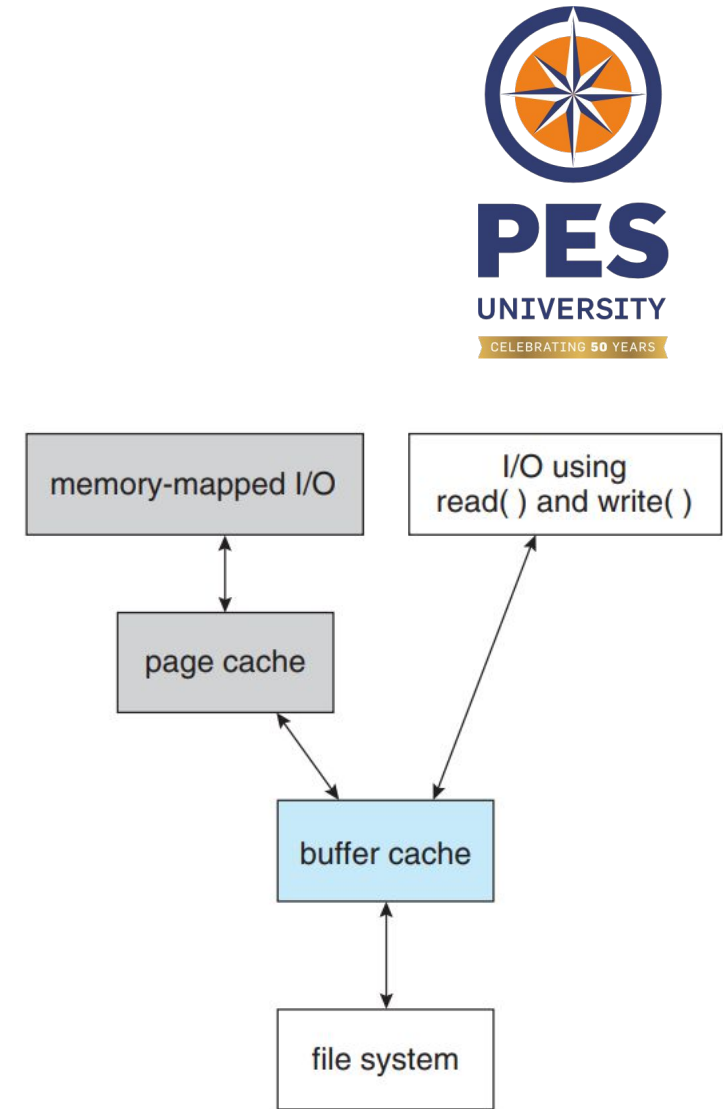
- Keeping data and metadata close together
- Buffer cache – separate section of main memory for frequently used blocks
- Free-behind and read-ahead – techniques to optimize sequential access
- Reads frequently slower than writes

- Some systems maintain a separate section of main memory for a **buffer cache**, where blocks are kept under the assumption that they will be used again shortly.
- Other systems **cache file data using a page cache**. The page cache uses virtual memory techniques to cache file data as pages rather than as file-system-oriented blocks.
- Caching file data using virtual addresses is far more efficient than caching through physical disk blocks, as accesses interface with virtual memory rather than the file system.
- Several systems—including Solaris, Linux, and Windows —use page caching to cache both process pages and file data. This is known as **unified virtual memory**.

OPERATING SYSTEMS

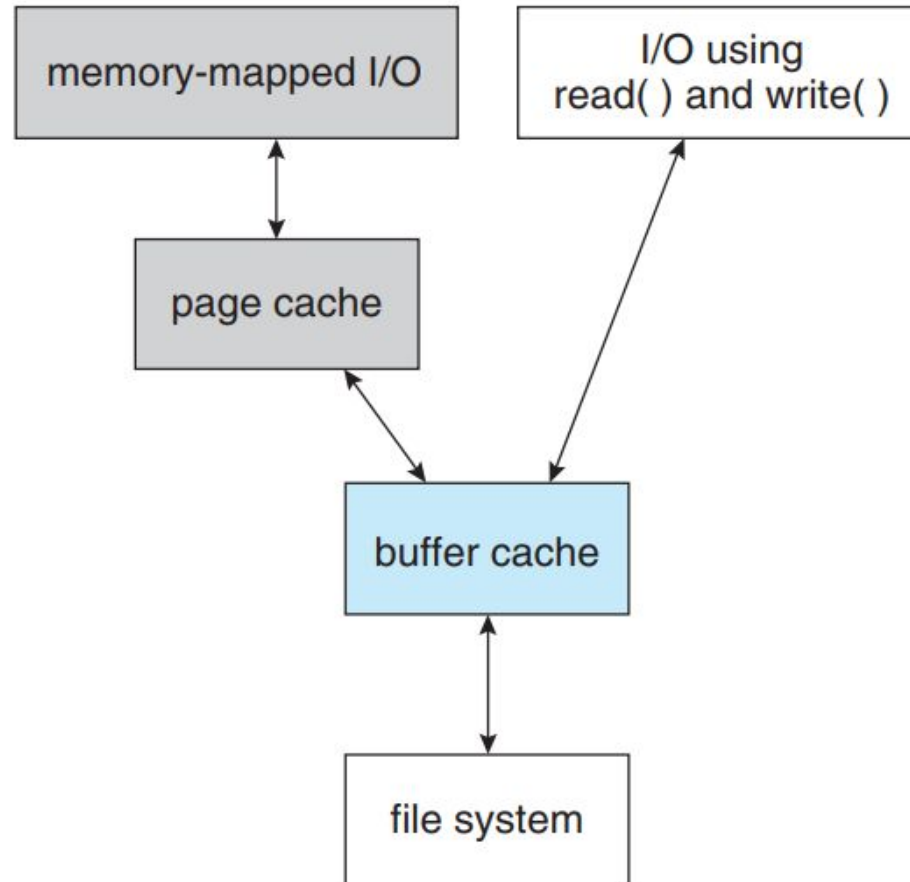
I/O without a unified buffer cache

- Consider standard system calls `read()` and `write()`
- Without a unified buffer cache, the `read()` and `write()` system calls go through the buffer cache.
- The memory-mapping call, requires two caches—the page cache and the buffer cache.
- A memory mapping proceeds by reading in disk blocks from the file system and storing them in the buffer cache. Because the virtual memory system does not interface with the buffer cache, the contents of the file in the buffer cache must be copied into the page cache.
- This situation, known as **double caching**, requires caching file-system data twice. Not only does it waste memory but it also wastes significant CPU and I/O cycles due to the extra data movement within system memory.



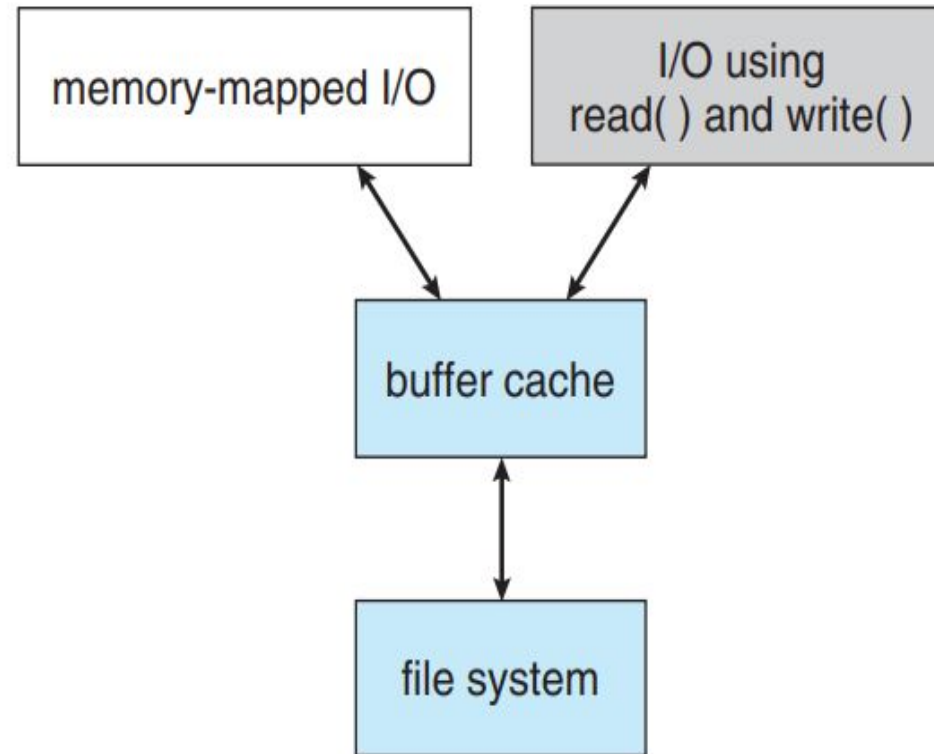
OPERATING SYSTEMS

I/O without a unified buffer cache



OPERATING SYSTEMS

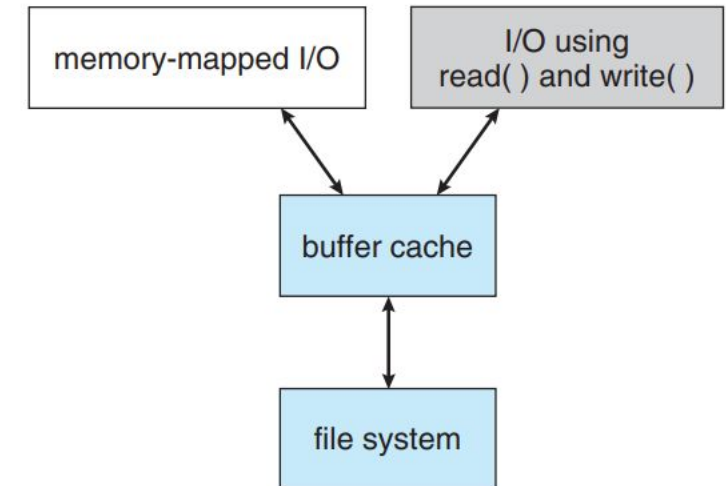
I/O using a unified buffer cache



OPERATING SYSTEMS

I/O using a unified buffer cache

- When a unified buffer cache is provided, both memory mapping and the read() and write() system calls use the same page cache.
- This has the benefit of avoiding double caching, and it allows the virtual memory system to manage file-system data



- Sequential access can be optimized by techniques known as free-behind and read-ahead.
- **Free-behind** removes a page from the buffer as soon as the next page is requested. The previous pages are not likely to be used again and waste buffer space.
- With **read-ahead**, a requested page and several subsequent pages are read and cached. These pages are likely to be requested after the current page is processed.
- Retrieving these data from the disk in one transfer and caching them saves a considerable amount of time



THANK YOU

Pavan A C

Department of Computer Science and Engineering

pavanac@pes.edu

OPERATING SYSTEMS

UE24CS242B



File Management

Pavan A C

Department of Computer Science & Engineering

OPERATING SYSTEMS



Mass-Storage Structure

Pavan A C

Department of Computer Science & Engineering

OPERATING SYSTEMS

Slides Credits for all PPTs of this course

- The slides/diagrams in this course are an **adaptation, combination, and enhancement** of material from the following resources and persons:

1. Slides of Operating System Concepts, Abraham Silberschatz, Peter Baer Galvin, Greg Gagne - 9th edition 2013 and some slides from 10th edition 2018
2. Some conceptual text and diagram from Operating Systems - Internals and Design Principles, William Stallings, 9th edition 2018
3. Some presentation transcripts from A. Frank – P. Weisberg
4. Some conceptual text from Operating Systems: Three Easy Pieces, Remzi Arpaci-Dusseau, Andrea Arpaci Dusseau



OPERATING SYSTEMS

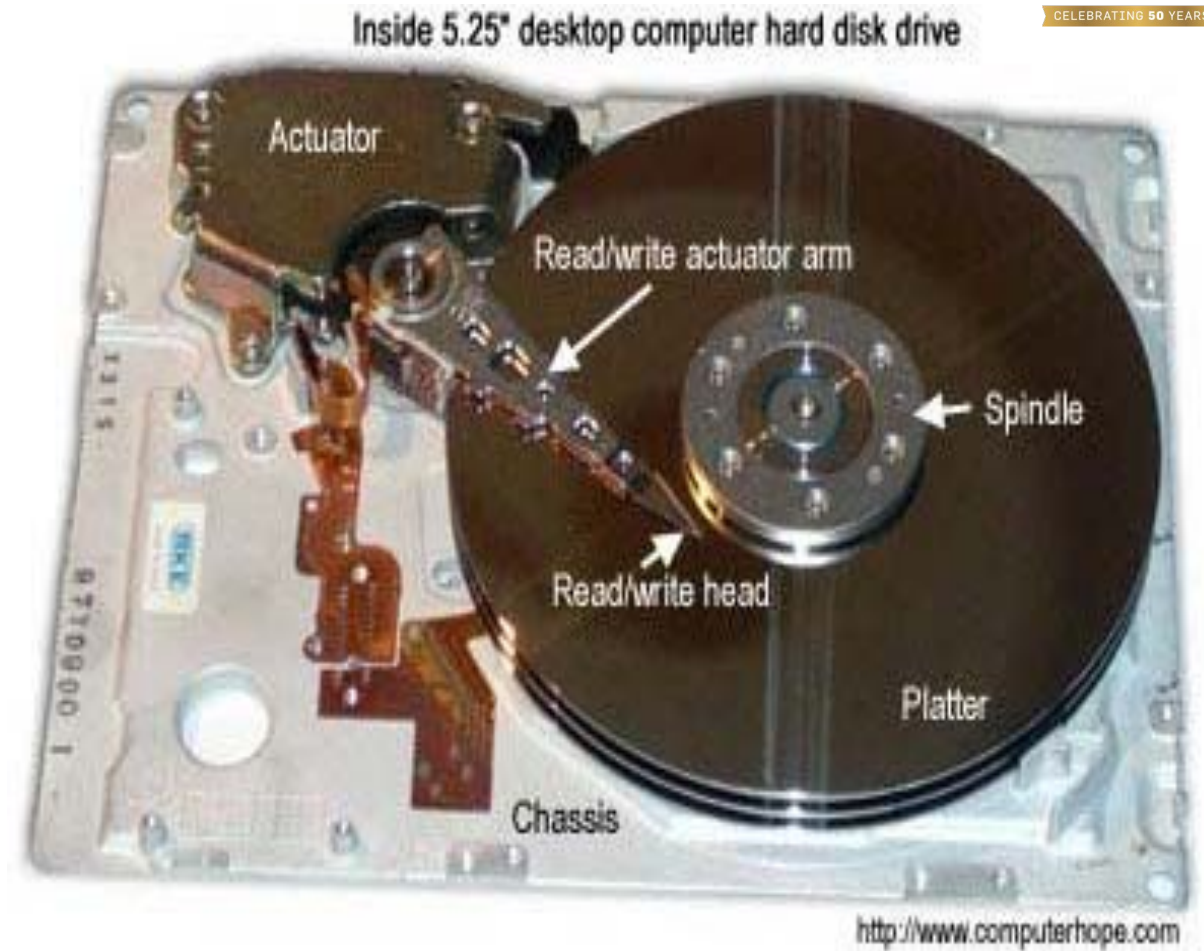
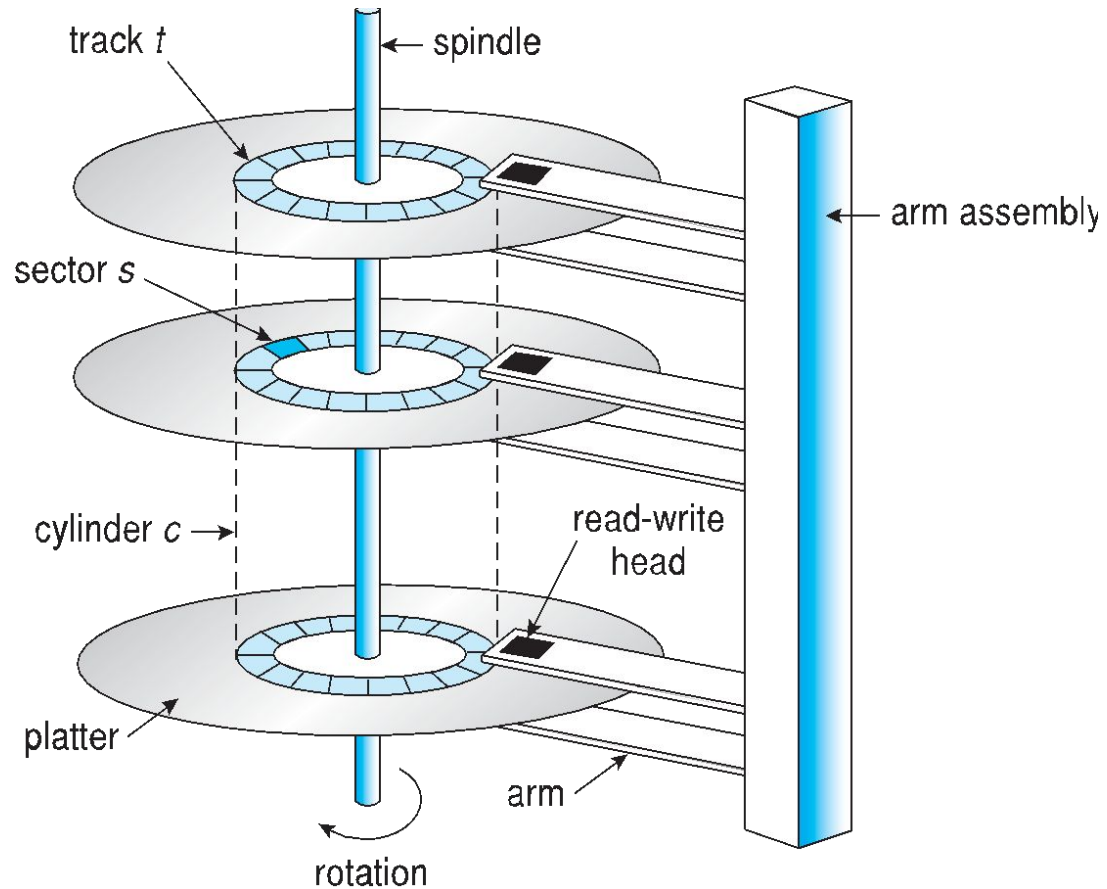
Overview of Mass Storage Structure

- **Magnetic disks** provide bulk of secondary storage of modern computers.
- Each disk platter has a flat circular shape, like a CD. Common platter diameters range from 1.8 to 3.5 inches.
- The two surfaces of a platter are covered with a magnetic material.
- The heads are attached to a disk arm that moves all the heads as a unit.
- The surface of a platter is logically divided into circular tracks, which are subdivided into sectors.
- The set of tracks that are at one arm position makes up a cylinder.



OPERATING SYSTEMS

Moving-head Disk Mechanism



PES
UNIVERSITY

CELEBRATING 50 YEARS

OPERATING SYSTEMS

Overview of Mass Storage Structure

- When the disk is in use, a drive motor spins it at high speed. Most drives rotate 60 to 250 times per second.

Disk speed has two parts.

- The transfer rate is the rate at which data flow between the drive and the computer.
- The positioning time, or random-access time, consists of two parts
 - The time necessary to move the disk arm to the desired cylinder, called the **seek time**.
 - the time necessary for the desired sector to rotate to the disk head, called the **rotational latency**.

Disk Latency = Seek time + Rotational latency + Transfer time



OPERATING SYSTEMS

Overview of Mass Storage Structure



- Disk head flies on an extremely thin cushion of air.
- Head will sometimes damage the magnetic surface when it makes contact with it.
 - Head crash
 - Not recoverable
- A disk can be **removable**.
- Removable magnetic disks generally consist of one platte
- Examples CDs, DVDs, and Blu-ray discs as well as removable flash-memory



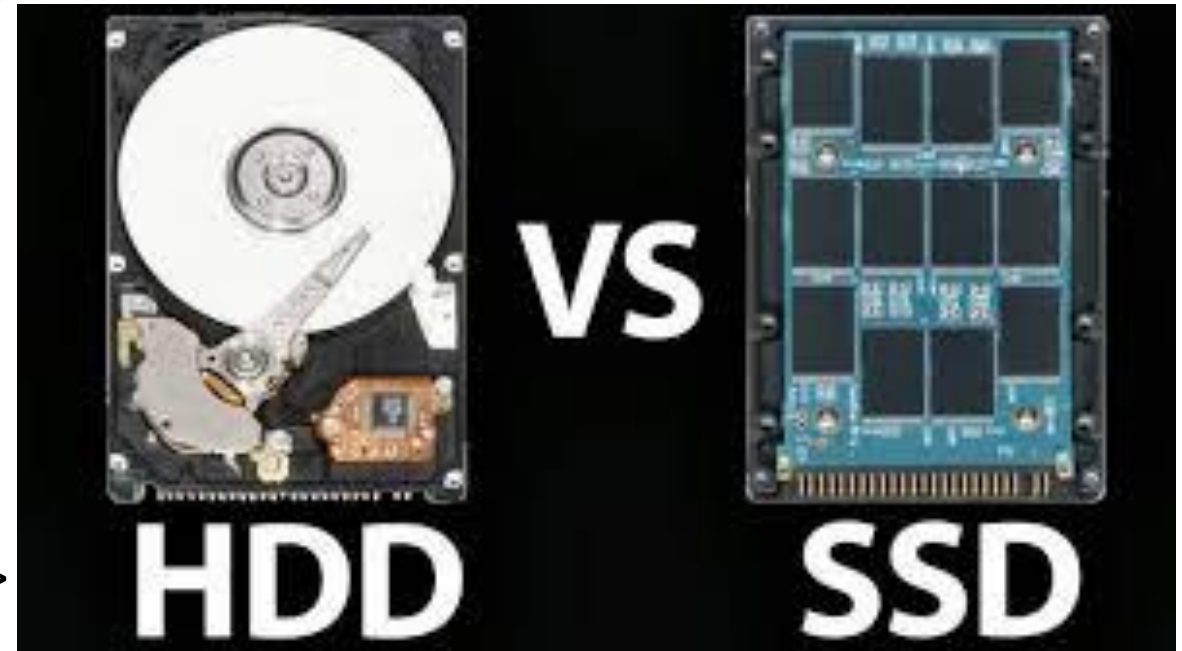
OPERATING SYSTEMS

Mass Storage Structure (Cont.)

- Drive attached to computer via **I/O bus**
 - Busses vary, including **EIDE**, **ATA**, **SATA**, **USB**, **Fiber Channel**, **SCSI**, **SAS**, **Firewire**
 - **Host controller** in computer uses bus to talk to **disk controller** built into drive or storage array



- Nonvolatile memory used like a hard drive
- Can be more reliable than HDDs
- More expensive per MB
- Maybe have shorter life span
- Less capacity
- But much faster
- Standard Bus interfaces can be too slow -> the system bus (PCI, for example)
- No moving parts, so no seek time or rotational latency



OPERATING SYSTEMS

Magnetic Tape

- Was early secondary-storage medium
- Relatively permanent and holds large quantities of data
- Access time slow
- Random access ~1000 times slower than disk
- Mainly used for backup, storage of infrequently-used data, transfer medium between systems





THANK YOU

Pavan A C

Department of Computer Science and Engineering

pavanac@pes.edu

OPERATING SYSTEMS

UE24CS242B



File Management

Pavan A C

Department of Computer Science & Engineering

OPERATING SYSTEMS

Mass-Storage Structure – Disk Scheduling
FCFS, SSTF, SCAN, C-SCAN, LOOK

Pavan A C

Department of Computer Science & Engineering

OPERATING SYSTEMS

Slides Credits for all PPTs of this course



- The slides/diagrams in this course are an **adaptation**, **combination**, and **enhancement** of material from the following resources and persons:

1. Slides of Operating System Concepts, Abraham Silberschatz, Peter Baer Galvin, Greg Gagne - 9th edition 2013 and some slides from 10th edition 2018
2. Some conceptual text and diagram from Operating Systems - Internals and Design Principles, William Stallings, 9th edition 2018
3. Some presentation transcripts from A. Frank – P. Weisberg
4. Some conceptual text from Operating Systems: Three Easy Pieces, Remzi Arpaci-Dusseau, Andrea Arpaci Dusseau

OPERATING SYSTEMS

Disk Scheduling

- The operating system is responsible for using hardware efficiently — for the disk drives, this means having a fast access time and disk bandwidth
- Minimize seek time
- Seek time $\approx$ seek distance
- Disk **bandwidth** is the total number of bytes transferred, divided by the total time between the first request for service and the completion of the last transfer



- There are many sources of disk I/O request
 - OS
 - System processes
 - Users processes
- I/O request includes input or output mode, disk address, memory address, number of sectors to transfer
- OS maintains queue of requests, per disk or device
- Idle disk can immediately work on I/O request, busy disk means work must queue
 - Optimization algorithms only make sense when a queue exists

OPERATING SYSTEMS

Disk Scheduling (Cont.)

- Note that drive controllers have small buffers and can manage a queue of I/O requests (of varying “depth”)
- Several algorithms exist to schedule the servicing of disk I/O requests
- The analysis is true for one or many platters
- We illustrate scheduling algorithms with a request queue (0-199)

98, 183, 37, 122, 14, 124, 65, 67

Head pointer 53



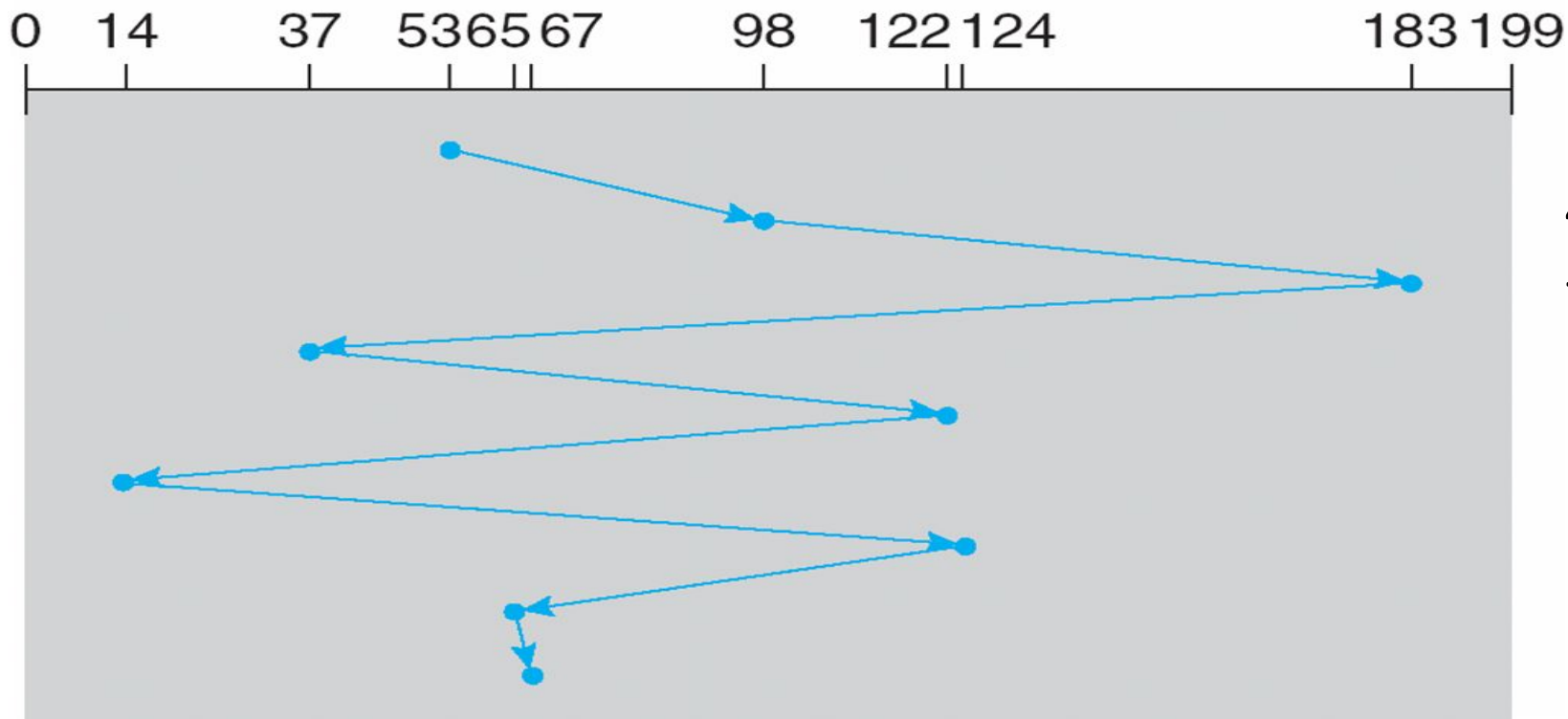
OPERATING SYSTEMS

FCFS

Illustration shows total head movement of 640 cylinders

queue = 98, 183, 37, 122, 14, 124, 65, 67

head starts at 53

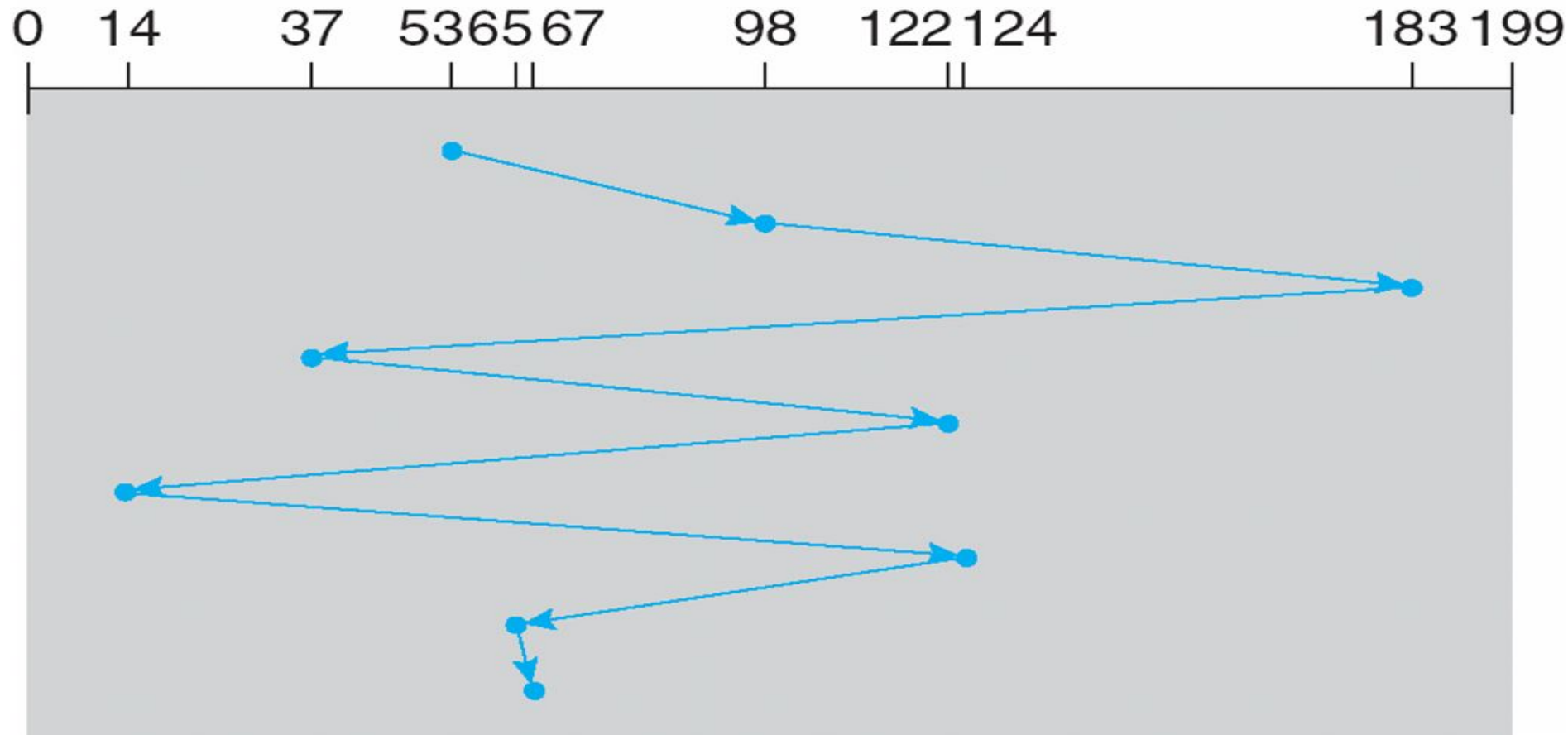


$$45 + 85 + 146 + 85 + 108 + 110 + 59 + 2 = 640 \text{ cylinders}$$

OPERATING SYSTEMS

FCFS

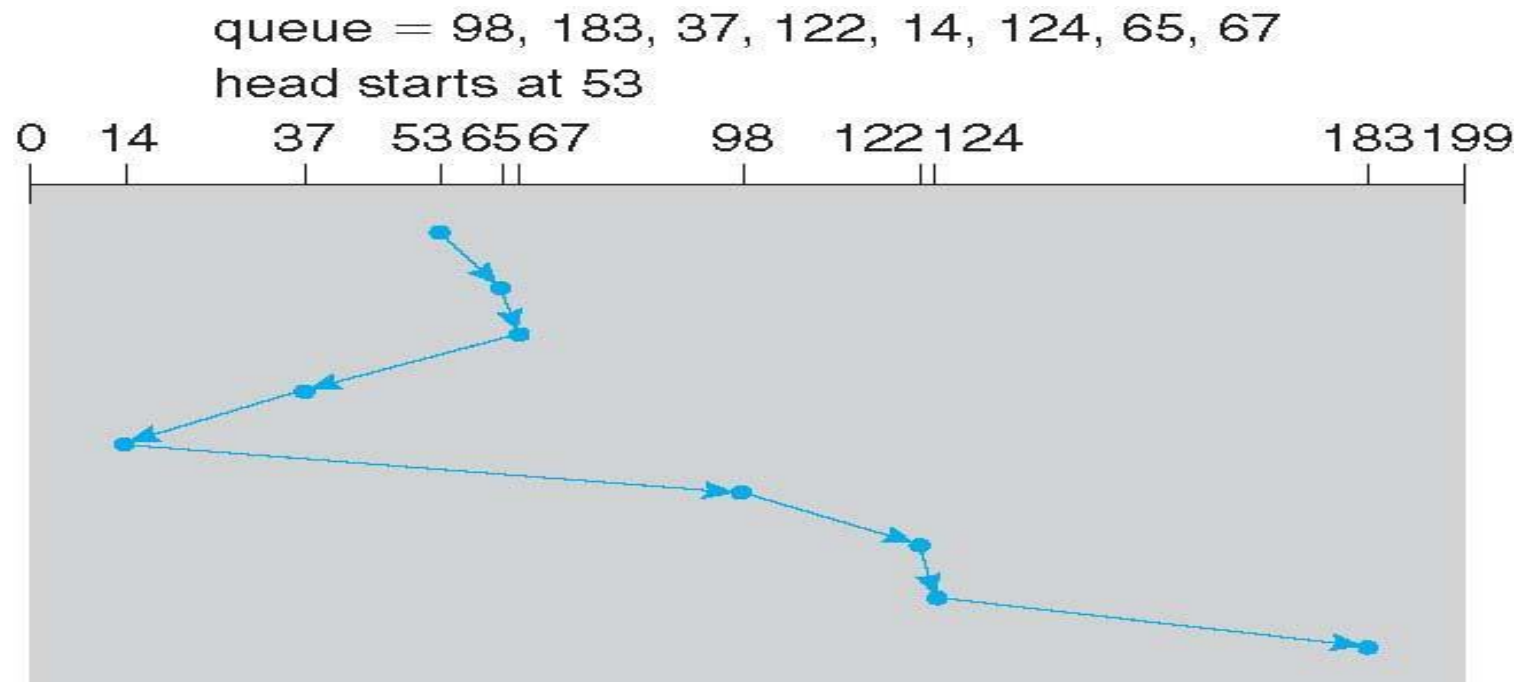
queue = 98, 183, 37, 122, 14, 124, 65, 67
head starts at 53



OPERATING SYSTEMS

SSTF

- Shortest Seek Time First selects the request with the minimum seek time from the current head position
- SSTF scheduling is a form of SJF scheduling; may cause starvation of some requests
- Illustration shows total head movement of 236 cylinders



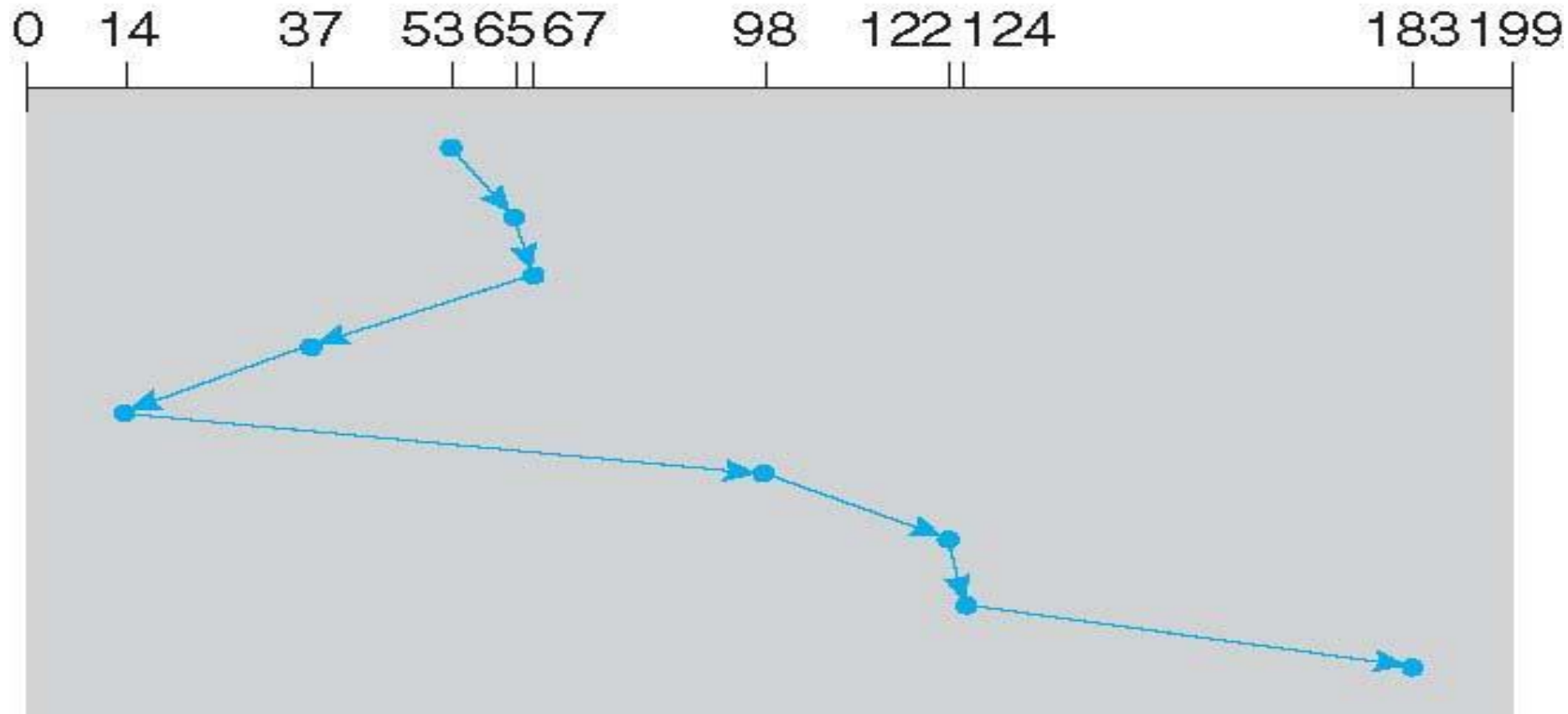
$$12+2+30+23+84+24+2+59 = 236 \text{ cylinders}$$

OPERATING SYSTEMS

SSTF

queue = 98, 183, 37, 122, 14, 124, 65, 67

head starts at 53



PES
UNIVERSITY

CELEBRATING 50 YEARS

- The disk arm starts at one end of the disk, and moves toward the other end, servicing requests until it gets to the other end of the disk, where the head movement is reversed and servicing continues.
- **SCAN algorithm** Sometimes called the **elevator algorithm**
- But note that if requests are uniformly dense, largest density at other end of disk and those wait the longest

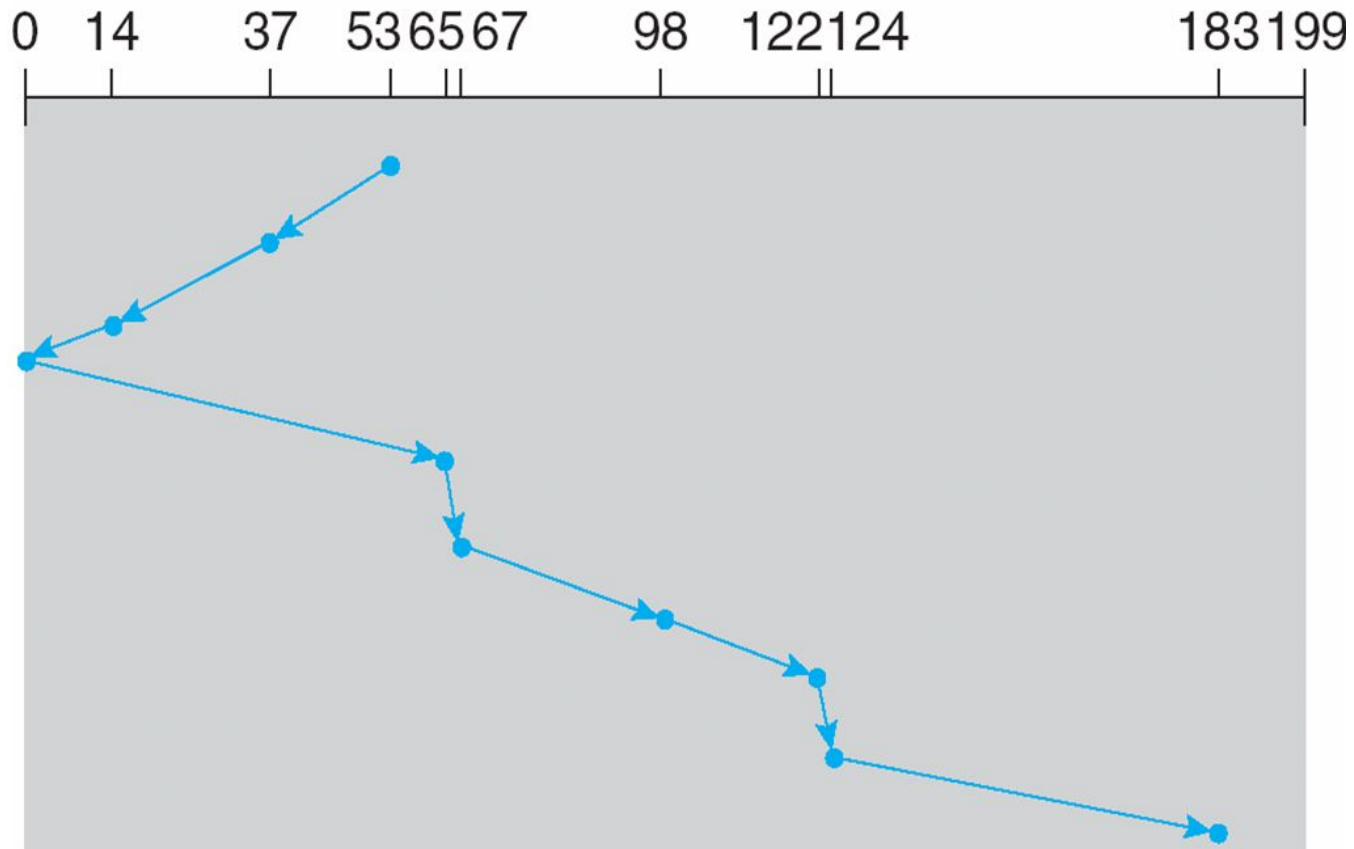
OPERATING SYSTEMS

SCAN (Cont.)



queue = 98, 183, 37, 122, 14, 124, 65, 67

head starts at 53



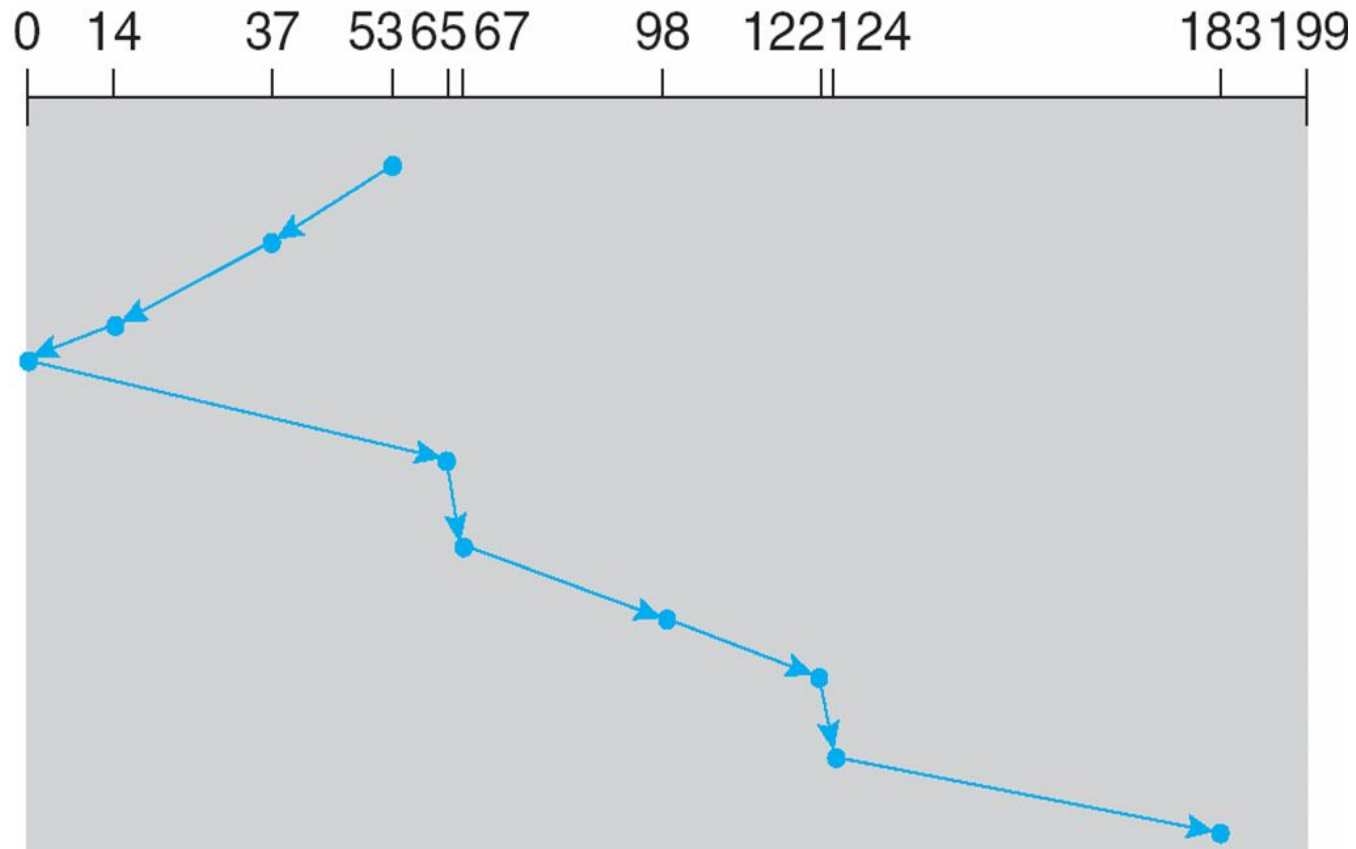
Number of cylinder moves=
 $16+23+79+2+31+24+2+59=236$

OPERATING SYSTEMS

SCAN (Cont.)

queue = 98, 183, 37, 122, 14, 124, 65, 67

head starts at 53

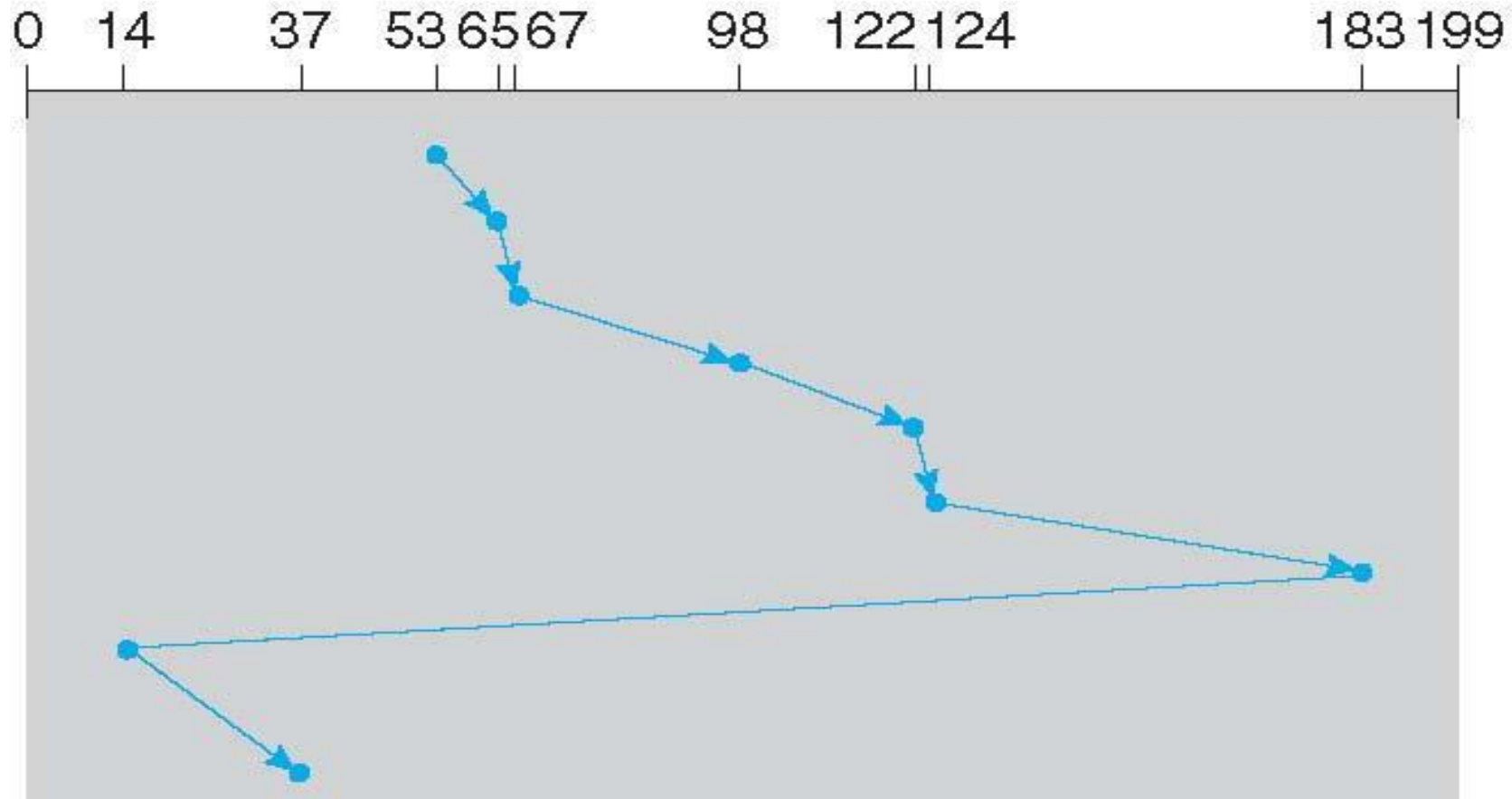


OPERATING SYSTEMS

C-LOOK (Cont.)

queue = 98, 183, 37, 122, 14, 124, 65, 67

head starts at 53



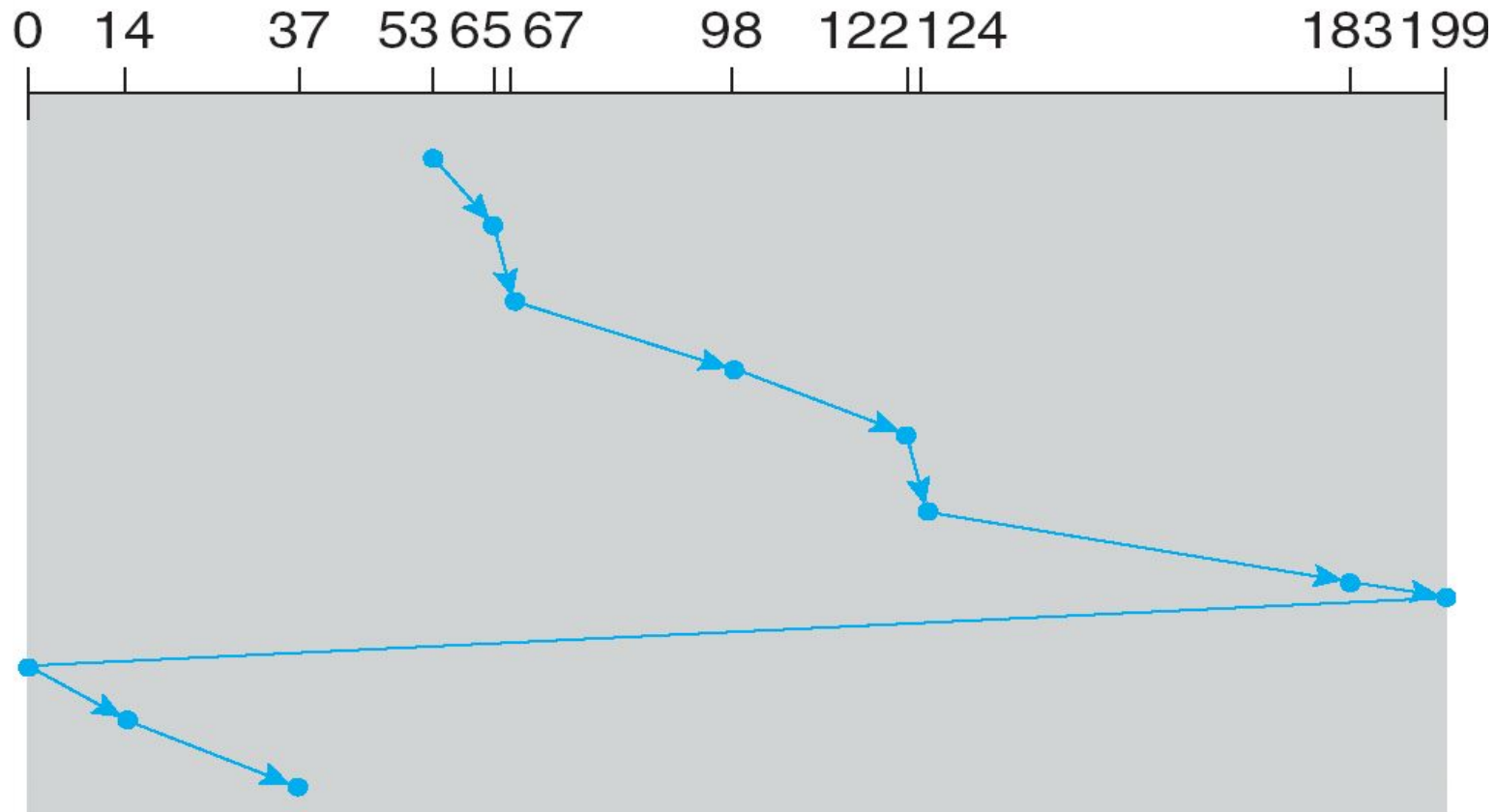
- Provides a more uniform wait time than SCAN
- The head moves from one end of the disk to the other, servicing requests as it goes
 - When it reaches the other end, however, it immediately returns to the beginning of the disk, without servicing any requests on the return trip
- Treats the cylinders as a circular list that wraps around from the last cylinder to the first one
- Total number of cylinders?

OPERATING SYSTEMS

C-SCAN (Cont.)

queue = 98, 183, 37, 122, 14, 124, 65, 67

head starts at 53

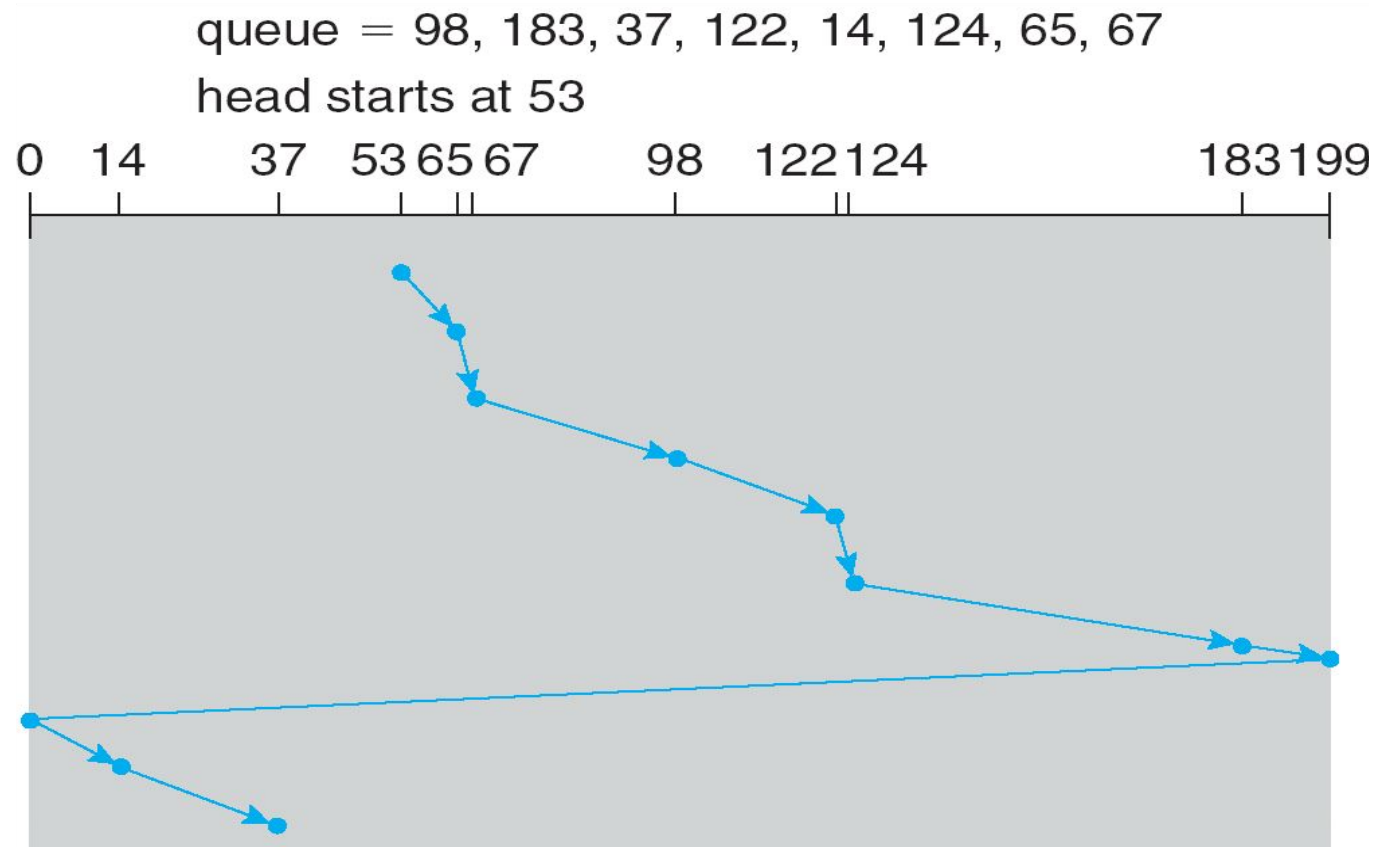


PES
UNIVERSITY

CELEBRATING 50 YEARS

OPERATING SYSTEMS

C-SCAN (Cont.)



$$\begin{aligned} &= (65-53) + (67-65) + (98-67) + (122-98) + (124-122) + (183-124) + (199-183) + (199-0) + (14-0) + (37-14) \\ &= 12 + 2 + 31 + 24 + 2 + 59 + 16 + 199 + 14 + 23 \\ &= 382 \end{aligned}$$

OPERATING SYSTEMS

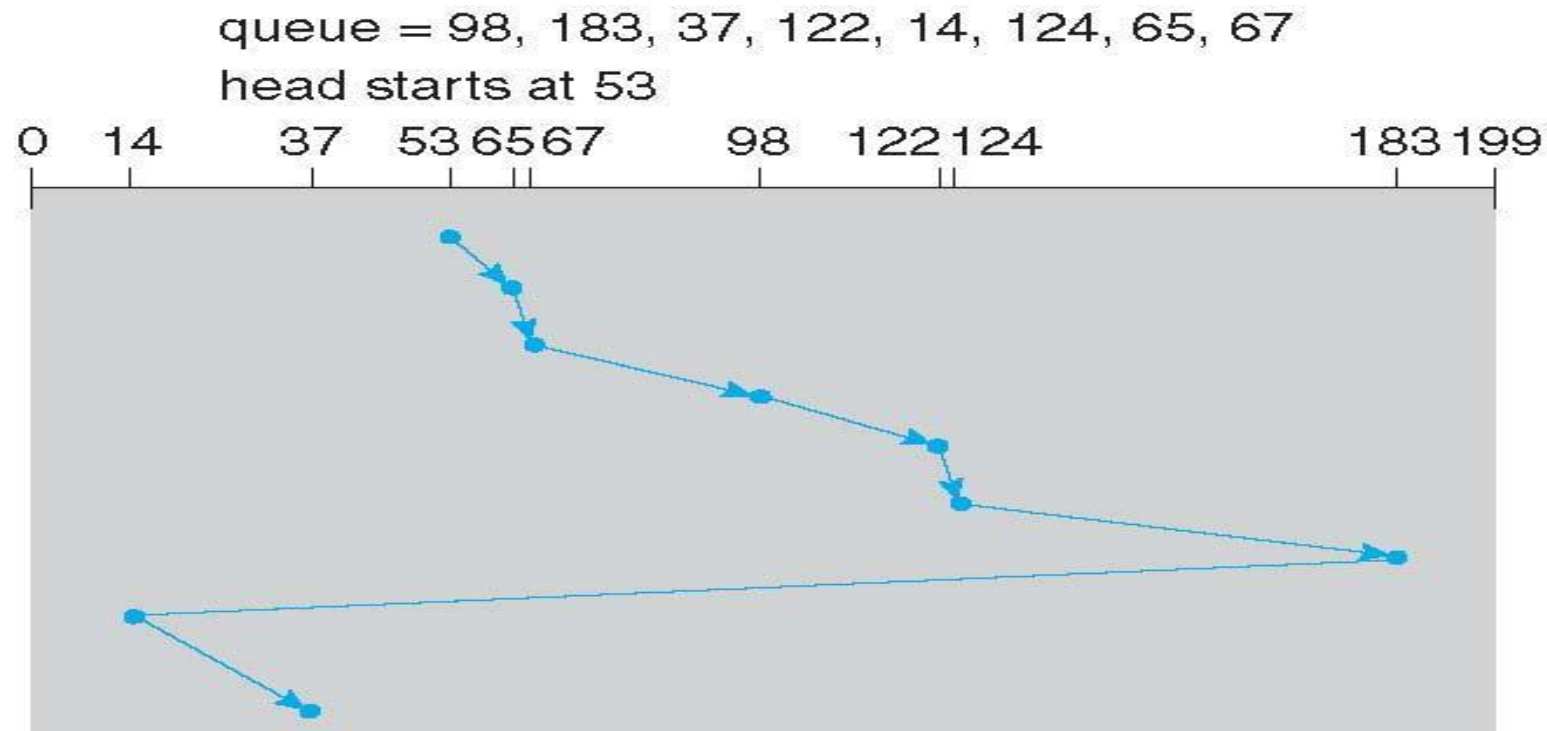
LOOK & C-LOOK

- LOOK a version of SCAN, C-LOOK a version of C-SCAN
- Arm only goes as far as the last request in each direction, then reverses direction immediately, without first going all the way to the end of the disk
- Total number of cylinders?



OPERATING SYSTEMS

C-LOOK (Cont.)



Total head movements incurred while servicing these requests

$$\begin{aligned} &= (65 - 53) + (67 - 65) + (98 - 67) + (122 - 98) + (124 - 122) + (183 - 124) + (183 - 14) + (37 - 14) \\ &= 12 + 2 + 31 + 24 + 2 + 59 + 169 + 23 \\ &= 322 \end{aligned}$$

- SSTF is common and has a natural appeal
- SCAN and C-SCAN perform better for systems that place a heavy load on the disk
 - Less starvation
- Performance depends on the number and types of requests
- Requests for disk service can be influenced by the file-allocation method
 - And metadata layout
- The disk-scheduling algorithm should be written as a separate module of the operating system, allowing it to be replaced with a different algorithm if necessary
- Either SSTF or LOOK is a reasonable choice for the default algorithm

- What about rotational latency?
 - Difficult for OS to calculate
 - The rotational latency can be nearly as large as the average seek time.
 - It is difficult for the operating system to schedule for improved rotational latency, though, because modern disks do not disclose the physical location of logical blocks.
 - Disk manufacturers have been alleviating this problem by implementing disk-scheduling algorithms in the controller hardware built into the disk drive.
- How does disk-based queueing effect OS queue ordering efforts?
 - If the OS sends a batch of requests to the controller, the controller can queue them and then schedule them to improve both the seek time and the rotational latency.



THANK YOU

Pavan A C

Department of Computer Science and Engineering

pavanac@pes.edu

OPERATING SYSTEMS

UE24CS242B



File Management

Pavan A C

Department of Computer Science & Engineering

OPERATING SYSTEMS



Mass-Storage Structure – RAID

Pavan A C

Department of Computer Science & Engineering

OPERATING SYSTEMS

Slides Credits for all PPTs of this course



- The slides/diagrams in this course are an **adaptation**, **combination**, and **enhancement** of material from the following resources and persons:
 1. Slides of Operating System Concepts, Abraham Silberschatz, Peter Baer Galvin, Greg Gagne - 9th edition 2013 and some slides from 10th edition 2018
 2. Some conceptual text and diagram from Operating Systems - Internals and Design Principles, William Stallings, 9th edition 2018
 3. Some presentation transcripts from A. Frank – P. Weisberg
 4. Some conceptual text from Operating Systems: Three Easy Pieces, Remzi Arpaci-Dusseau, Andrea Arpaci Dusseau

OPERATING SYSTEMS

RAID Basics

- ❑ RAID is a Disk Organization technique
 - ❑ Addresses performance and reliability issues
 - ❑ Multiple disk drives provide (or improve) reliability via **redundancy**
- ❑ Many systems today need to store many terabytes of data
- ❑ Don't want to use single, large disk
 - ❑ too expensive
 - ❑ failures could be catastrophic
- ❑ Would prefer to use many smaller disks
- ❑ **Redundant Array of Independent (inexpensive) Disks**



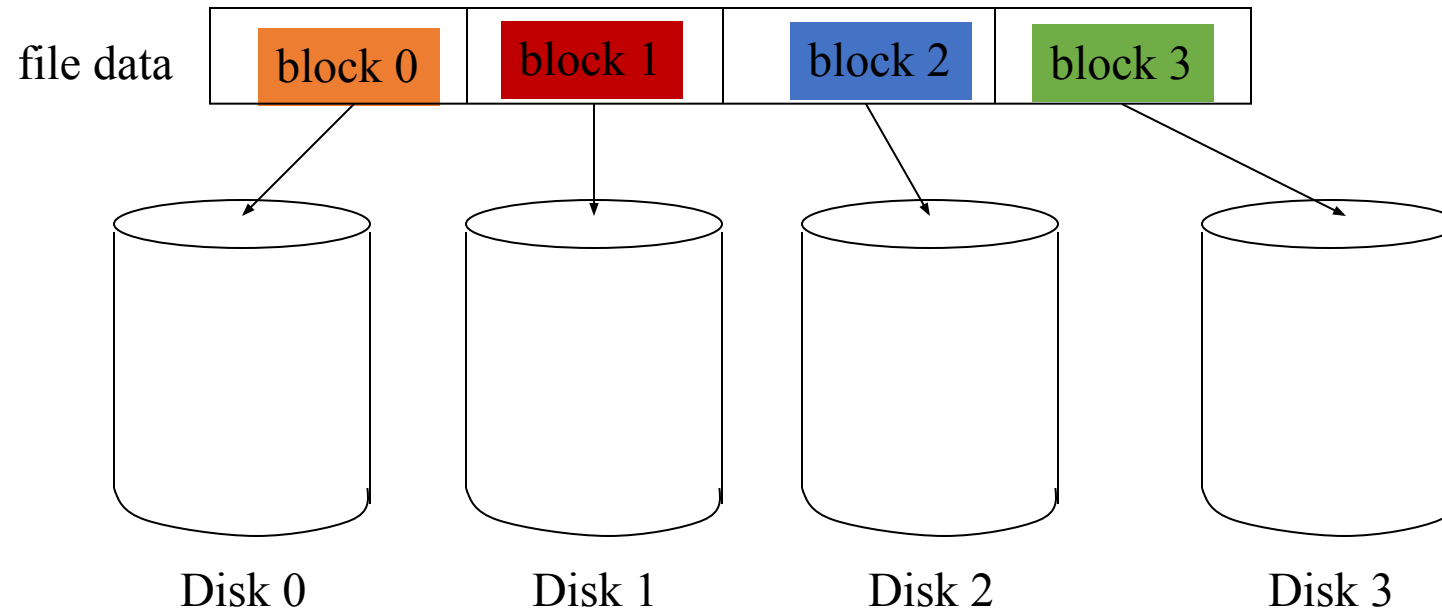
OPERATING SYSTEMS

RAID Basics (Cont.)

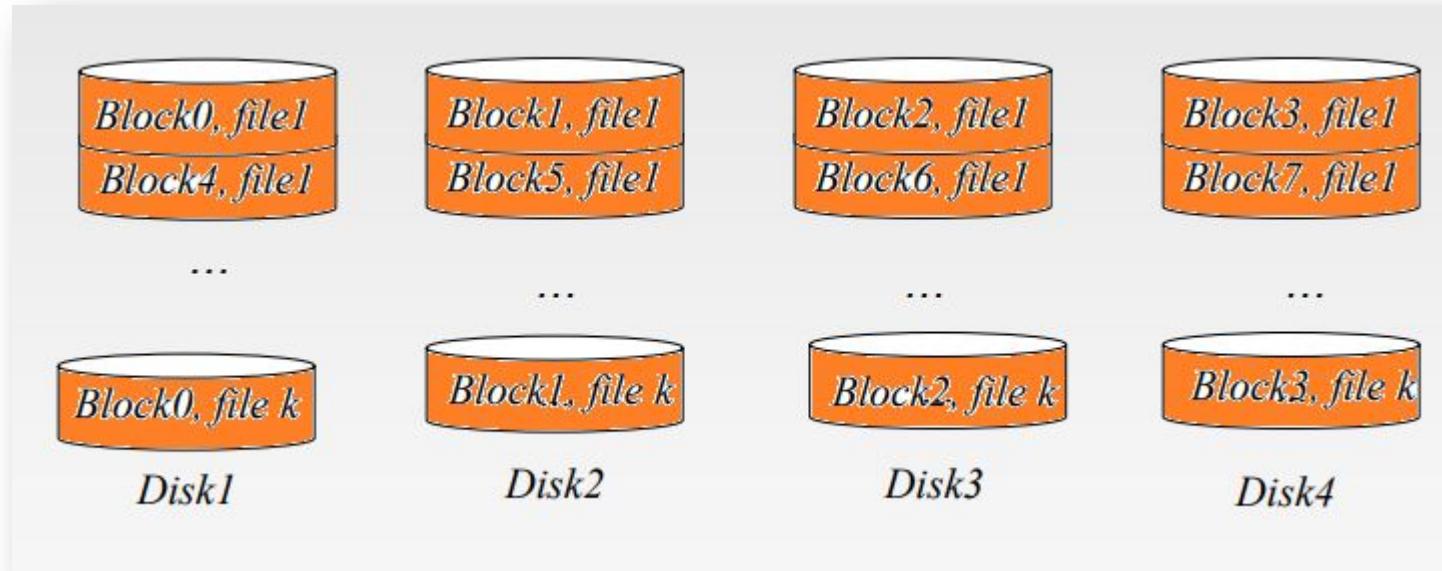
- ☐ Basic idea is to connect multiple disks together to provide
 - ☐ large storage capacity
 - ☐ faster access to reading data
 - ☐ redundant data
- ☐ Many different levels of RAID systems
 - ☐ differing levels of redundancy, error checking, capacity, and cost



- Take file data and map it to different disks
- Allows for reading data in parallel
- Transfer rate is improved by striping data across the disks
 - **Bit-level striping and block-level striping (most common)**



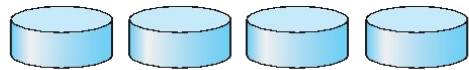
- Requests for different blocks can run in parallel if the blocks reside on different disks
- A request for a long sequence of blocks can utilize all disks in parallel



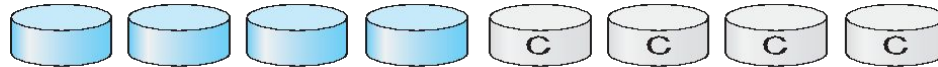
- Keep two copies of data on two separate disks
- Gives good error recovery
 - if some data is lost, get it from the other source
- Expensive
 - requires twice as many disks
- Write performance can be slow
 - have to write data to two different spots
- Read performance is enhanced
 - can read data from file in parallel

- **Mean time to failure**
 - If MTTF of a single disk is 100,000 hours, MTTF of some disk in an array of 100 disks = $100000/100 = 1000$ hours or 41.66 days
- **Mean time to repair** – time taken to replace a failed disk and to restore the data on it
- **Mean time to data loss** based on above factors
 - If mirrored disks fail independently (i.e., not related to power failures and natural disasters), consider disk with MTTF of 100,000 hours and MTTR is 10 hours

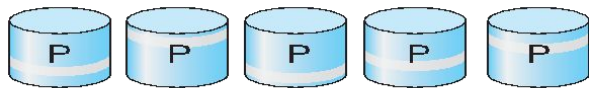
- Several improvements in disk-use techniques involve the use of multiple disks working cooperatively
- Disk **striping** uses a group of disks as one storage unit
- RAID is arranged into six different levels
- RAID schemes improve performance and improve the reliability of the storage system by storing redundant data
 - **Mirroring** or **shadowing** (**RAID 1**) keeps duplicate of each disk
 - Striped mirrors (**RAID 1+0**) or mirrored stripes (**RAID 0+1**) provides high performance and high reliability
 - **Block interleaved parity** (**RAID 4, 5, 6**) uses much less redundancy



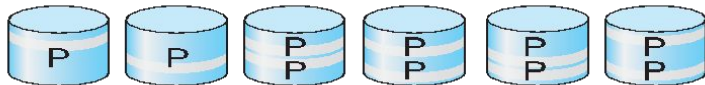
(a) RAID 0: non-redundant striping.



(b) RAID 1: mirrored disks.



(f) RAID 5: block-interleaved distributed parity.



(a) RAID 6: P + Q redundancy.

- P indicates error-correcting bits
- C indicates a second copy of the data
- P + Q redundancy scheme uses 2 parity values P and Q

OPERATING SYSTEMS

RAID Levels 0 and 1

❑ **RAID level 0** refers to disk arrays with striping at the level of blocks

- ❑ No redundancy (such as mirroring or parity bits)
- ❑ lots of disks means low Mean Time To Failure (MTTF)



❑ **RAID level 1** refers to disk mirroring.

- ❑ A complete file is stored on a single disk
- ❑ A second disk contains an exact copy of the file
- ❑ Provides complete redundancy of data
- ❑ file data can be read in parallel
- ❑ must write the data out twice
- ❑ Most expensive RAID implementation
- ❑ requires twice as much storage space



- ❑ **RAID level 5** stripes file data and checks data over all the disks
 - ❑ no longer a single check disk
- ❑ Consists of block-level striping with distributed parity.
 - ❑ Unlike RAID 4, parity information is distributed among the drives
- ❑ Drastically improves the performance of multiple writes
 - ❑ they can now be done in parallel
- ❑ read and write performance close to that of RAID Level-1

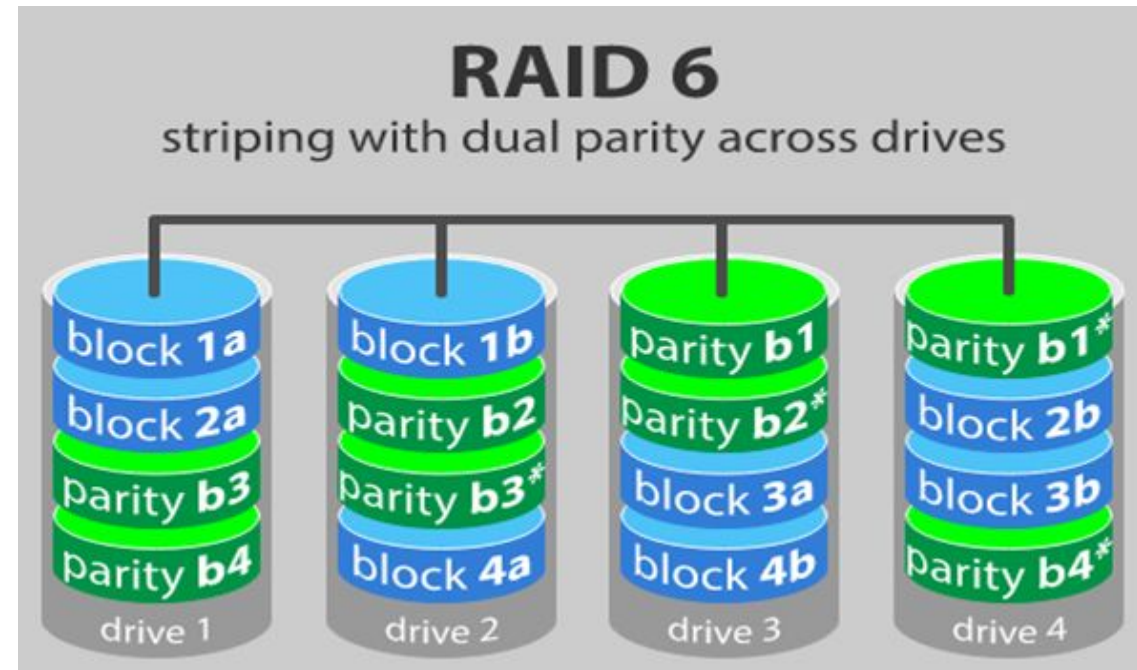


(f) RAID 5: block-interleaved distributed parity.

OPERATING SYSTEMS

RAID Level-6

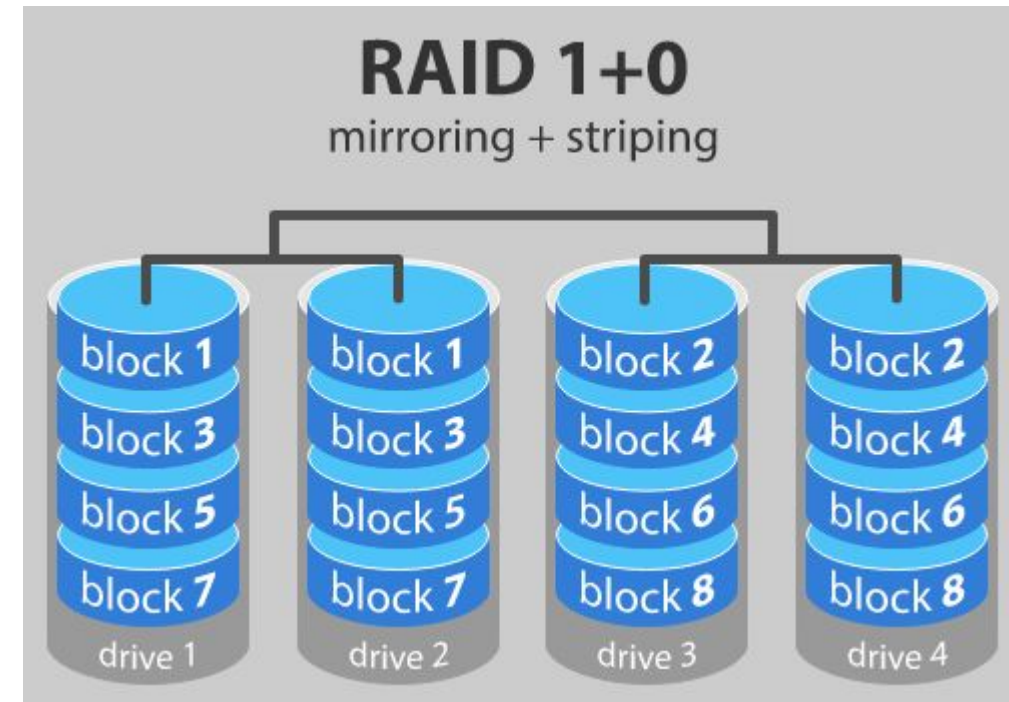
- ❑ RAID 6 is like RAID 5, but the parity data are written to two drives.
- ❑ Write data transactions are slower than RAID 5 due to the additional parity data that have to be calculated.



OPERATING SYSTEMS

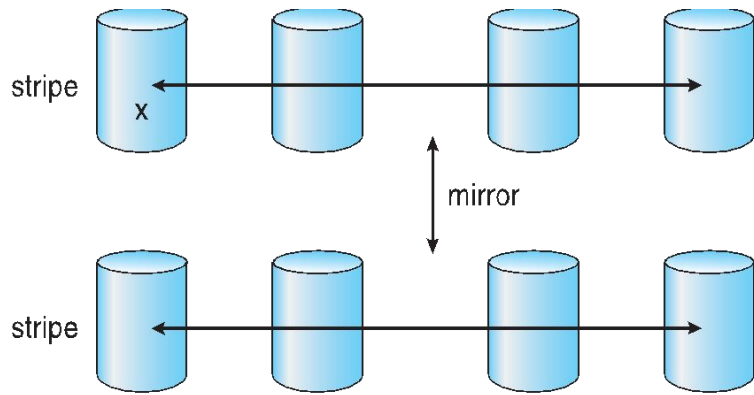
RAID level 10 - combining RAID 1 and RAID 0

- ❑ Provides security by mirroring all data on secondary drives while using striping across each set of drives to speed up data transfers.
- ❑ Rebuild time is very fast
- ❑ Half of the storage capacity goes to mirroring
 - ❑ so compared to large RAID 5 or RAID 6 arrays, this is an expensive way to have redundancy.

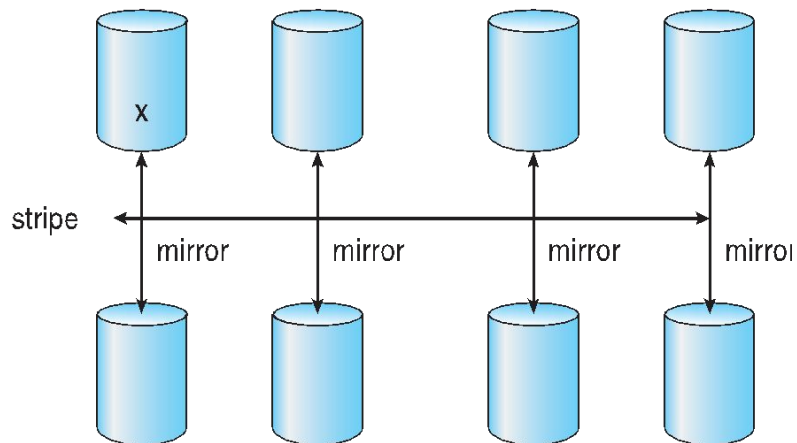


OPERATING SYSTEMS

RAID (0 + 1) and (1 + 0)



a) RAID 0 + 1 with a single disk failure.



b) RAID 1 + 0 with a single disk failure.

Disks are striped and then the stripe is mirrored to another, equivalent stripe. If a single disk fails, an entire stripe is inaccessible.

Disks are mirrored in pairs and then the resulting mirrored pairs are striped. This is better than 0+1 when a single disk fails



THANK YOU

Pavan A C

Department of Computer Science and Engineering

pavanac@pes.edu

OPERATING SYSTEMS

UE24CS242B



File Management

Pavan A C

Department of Computer Science & Engineering

OPERATING SYSTEMS



Protection

Pavan A C

Department of Computer Science & Engineering

OPERATING SYSTEMS

Slides Credits for all PPTs of this course

- The slides/diagrams in this course are an **adaptation, combination, and enhancement** of material from the following resources and persons:
1. Slides of Operating System Concepts, Abraham Silberschatz, Peter Baer Galvin, Greg Gagne - 9th edition 2013 and some slides from 10th edition 2018
 2. Some conceptual text and diagram from Operating Systems - Internals and Design Principles, William Stallings, 9th edition 2018
 3. Some presentation transcripts from A. Frank – P. Weisberg
 4. Some conceptual text from Operating Systems: Three Easy Pieces, Remzi Arpaci-Dusseau, Andrea Arpaci Dusseau



- Protection is a mechanism to control access of:
 - Users
 - Processes
 - Programs
- It ensures safe and controlled use of system resources
- Provides:
 - Policy definition
 - Enforcement mechanism
- Important in multi-user and shared systems

OPERATING SYSTEMS

Protection - Why is it needed?

- Prevent unauthorized access
- Avoid intentional misuse
- Protect system from faulty programs
- Enable safe resource sharing
- Improve system reliability



OPERATING SYSTEMS

Goals of protection

- Ensure authorized access only
- Prevent malicious violations
- Provide controlled resource usage
- Detect errors early
- Support flexible protection policies



OPERATING SYSTEMS

Principles of Protection (Least Privilege)

- Give minimum access rights required
- Reduce damage due to:
 - Bugs
 - Attacks
- Privileges should be:
 - Enabled when needed
 - Disabled when not needed



OPERATING SYSTEMS

Principles of Protection (Additional Protection Principles)

- Maintain audit trails (logs)
- Use separate user accounts
- Implement Role-Based Access Control (RBAC)
- Restrict:
 - Services
 - Access time
 - Remote connections



OPERATING SYSTEMS

Domain of protection



A computer system consists of:

- **Processes** (active entities)
- **Objects** (resources)

Objects include:

- **Hardware:** CPU, memory, printers, disks
- **Software:** files, programs, semaphores

Key Idea

- Each object has:
 - A **unique name**
 - A set of **allowed operations**
- A process can access an object **only through defined operations**

Access Control

- A process should access **only authorized resources**
- At any time, it should use **only required resources**

Principle: Need-to-Know

- Give **only necessary permissions**
- Limits:
 - Damage from errors
 - Security risks

OPERATING SYSTEMS

Domain structure

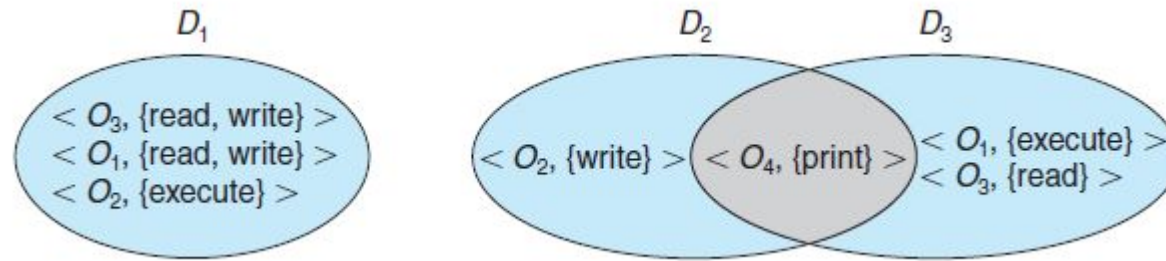
- A protection domain defines what resources a process can access
- Each domain contains:
 - Objects (files, memory, devices, etc.)
 - Access rights (read, write, execute, etc.)
- Access right is represented as:
 <object, set of operations>
- Example:
 <File F, {read, write}>
- A process can perform only the operations allowed in its domain
- Domains may share access rights



- A process executes within a domain:
 - Static domain
 - Fixed access rights throughout execution
 - Simple but may violate least privilege principle
 - Dynamic domain
 - Access rights can change during execution
 - Supports need-to-know principle
- Domain Switching
 - Process can switch between domains
 - Helps provide only required permissions at each phase
- If domain content cannot be modified:
 - Create a new domain and switch to it

OPERATING SYSTEMS

Domain structure example



- We have three domains: D_1 , D_2 , and D_3 .
- The access right $\langle O_4, \{\text{print}\} \rangle$ is shared by D_2 and D_3 , implying that a process executing in either of these two domains can print object O_4 .
- A process must be executing in domain D_1 to read and write object O_1 , while only processes in domain D_3 may execute object O_1 .

A protection domain can be implemented in different ways:

1. User as Domain

- Each user has a separate domain
- Access depends on **user identity**
- Domain switching occurs when:
 - User logs in / logs out

2. Process as Domain

- Each process has its own domain
- Access depends on **process identity**
- Domain switching occurs when:
 - One process communicates with another

3. Procedure as Domain

- Each procedure defines a domain
- Access limited to **local variables/resources**
- Domain switching occurs during:
 - **Procedure calls**

- In **UNIX**, a domain is associated with a **user**
- Each process runs with a **User ID (UID)**
- Access rights depend on the **current UID**

Domain Switching in UNIX

- Domain switching = **changing User ID temporarily**
- Achieved using a special file permission:
 - **setuid (Set User ID bit)**

Working of setuid

- When a file is executed:
 - **setuid OFF** → process runs with **current user's UID**
 - **setuid ON** → process runs with **file owner's UID**

Example

- User **A** runs a program owned by **B**:
 - setuid OFF → runs as **A**
 - setuid ON → runs as **B** (temporary privilege)
- After execution:
 - UID returns back to original user



THANK YOU

Pavan A C

Department of Computer Science and Engineering

pavanac@pes.edu

OPERATING SYSTEMS

UE24CS242B



File Management

Pavan A C

Department of Computer Science & Engineering

OPERATING SYSTEMS



Access Matrix

Pavan A C

Department of Computer Science & Engineering

OPERATING SYSTEMS

Slides Credits for all PPTs of this course

- The slides/diagrams in this course are an **adaptation, combination, and enhancement** of material from the following resources and persons:

1. Slides of Operating System Concepts, Abraham Silberschatz, Peter Baer Galvin, Greg Gagne - 9th edition 2013 and some slides from 10th edition 2018
2. Some conceptual text and diagram from Operating Systems - Internals and Design Principles, William Stallings, 9th edition 2018
3. Some presentation transcripts from A. Frank – P. Weisberg
4. Some conceptual text from Operating Systems: Three Easy Pieces, Remzi Arpaci-Dusseau, Andrea Arpaci Dusseau



OPERATING SYSTEMS

Access Matrix

- Protection model represented as a **matrix**
- **Rows** → **Domains**
- **Columns** → **Objects**
- Each cell contains **access rights**

access(i, j):

- Operations allowed on object **O_j** by domain **D_i**



OPERATING SYSTEMS

Access Matrix example

- Domains: **D1, D2, D3, D4**
- Objects: **F1, F2, F3, Printer**

Key Points:

- **D1** → Read F1, F3
- **D4** → Same as D1 + Write F1, F3
- **D2** → Access Printer only

object domain	F_1	F_2	F_3	printer
D_1	read		read	
D_2				print
D_3		read	execute	
D_4	read write		read write	

Ensures:

- Only **allowed access**
- Strict control of resources

OS decides:

- Which domain a process runs in
- What rights are given

Domain Switching

- Domain is treated as an **object**
- Switching = performing **switch operation**

Rule:

- Process can switch from **$D_i \rightarrow D_j$**
only if 'switch' right exists

- D2 → can switch to D3, D4
- D4 → can switch to D1
- D1 → can switch to D2

Additional Rights:

- **copy** → duplicate access rights
- **owner** → full control
- **control** → modify rights

domain \ object	F_1	F_2	F_3	laser printer	D_1	D_2	D_3	D_4
D_1	read		read			switch		
D_2				print			switch	switch
D_3		read	execute					
D_4	read write		read write		switch			

OPERATING SYSTEMS

Implementation of the Access Matrix – Global table

- Represented as: **<domain, object, rights>**
- System searches table to check permission

Drawbacks:

- Large size → not memory efficient
- Slow lookup (search required)
- Difficult to manage grouping



OPERATING SYSTEMS

Implementation of the Access Matrix – Access list for objects

- Each object has a list:
 - **(domain, rights)**

Advantages:

- Easy to check **who can access object**
- Supports default permissions

Limitation:

- Hard to know what a domain can access



OPERATING SYSTEMS

Role based access control

- Access permissions are assigned to **roles**, not directly to users
- A **role** represents a job/function (e.g., Admin, Faculty, Student)

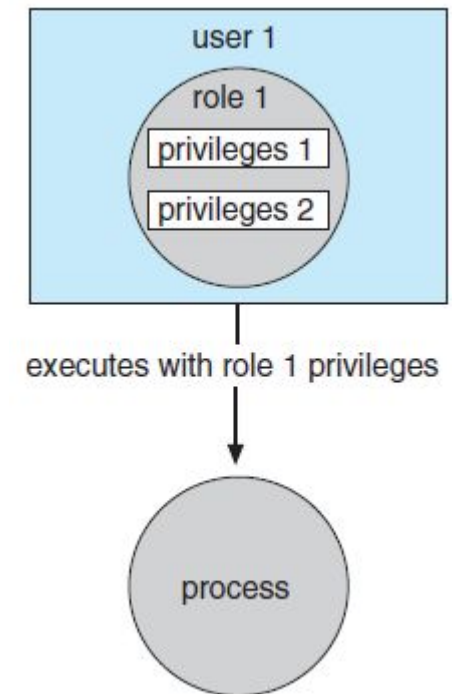
How it works:

- **Users** → assigned to roles
- **Roles** → assigned permissions
- Users gain access **through roles**

Example:

- **Admin** → read, write, delete
- **Faculty** → read, write
- **Student** → read only

If a user is assigned “Faculty” role → gets those permissions



OPERATING SYSTEMS

Role based access control

Advantages:

- Reduces complexity in access management
- Easy to update permissions (change role, not every user)
- Improves security (no direct permission assignment)
- Supports large systems efficiently

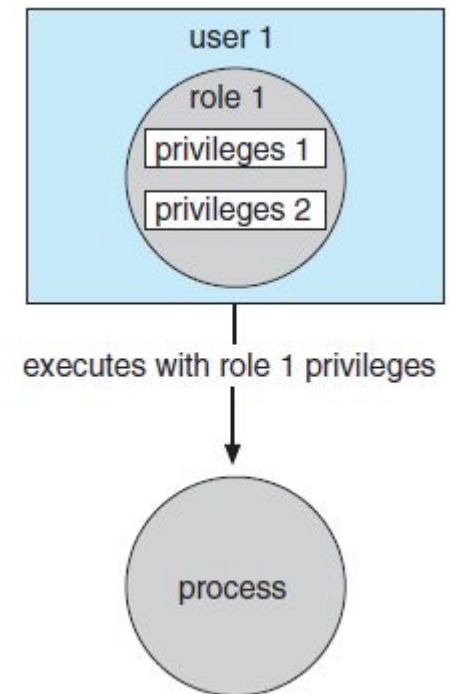
Limitation:

- Requires proper role design
- Too many roles → becomes complex



PES
UNIVERSITY

CELEBRATING 50 YEARS



OPERATING SYSTEMS

Revocation of access rights

- **Revocation** = Removing access rights from users/domains
- Needed in **dynamic protection systems**

Types of Revocation:

- **Immediate / Delayed**
 - Instant removal vs later
- **Selective / General**
 - Specific users vs all users
- **Partial / Total**
 - Some rights vs all rights
- **Temporary / Permanent**
 - Can be regained vs removed forever



Access List (ACL):

- Easy to revoke:
 - Simply **remove entry from list**
- Supports:
 - All types (selective, partial, etc.)

Capability List:

- Difficult to revoke:
 - Capabilities are **distributed across system**

Revocation Techniques:

- **Reacquisition**
 - Capabilities expire → must request again
- **Back-pointers**
 - Track all capabilities → modify them
- **Indirection**
 - Use global table → delete entry
- **Keys**
 - Change key → invalidate all old access



THANK YOU

Pavan A C

Department of Computer Science and Engineering

pavanac@pes.edu

OPERATING SYSTEMS

UE24CS242B



File Management

Pavan A C

Department of Computer Science & Engineering

OPERATING SYSTEMS



Security

Pavan A C

Department of Computer Science & Engineering

OPERATING SYSTEMS

Slides Credits for all PPTs of this course

- The slides/diagrams in this course are an **adaptation, combination, and enhancement** of material from the following resources and persons:
1. Slides of Operating System Concepts, Abraham Silberschatz, Peter Baer Galvin, Greg Gagne - 9th edition 2013 and some slides from 10th edition 2018
 2. Some conceptual text and diagram from Operating Systems - Internals and Design Principles, William Stallings, 9th edition 2018
 3. Some presentation transcripts from A. Frank – P. Weisberg
 4. Some conceptual text from Operating Systems: Three Easy Pieces, Remzi Arpaci-Dusseau, Andrea Arpaci Dusseau



Security ensures:

- Protection from **unauthorized access**
- Protection from **malicious attacks & accidental errors**

Covers:

- Data, programs, CPU, memory, network

Threat vs Attack

- **Threat** → potential danger
- **Attack** → actual attempt to break security

OPERATING SYSTEMS

Security Threats

- **Breach of confidentiality.** This type of violation involves unauthorized reading of data (or theft of information). Typically, a breach of confidentiality is the goal of an intruder
 - **Breach of integrity.** This violation involves unauthorized modification of data.
 - **Breach of availability.** This violation involves unauthorized destruction of data.
 - **Theft of service.** This violation involves unauthorized use of resources.
 - **Denial of service.** This violation involves preventing legitimate use of the system.
-
- **Masquerading** → pretending to be someone else
 - **Replay attack** → resending valid data
 - **Man-in-the-Middle** → intercepting communication



OPERATING SYSTEMS

Security measures

To protect a system, we must take security measures at four levels:

1. **Physical** → secure hardware & location
2. **Human** → proper user authorization
3. **Operating System** → internal protection
4. **Network** → secure communication



Program Threats

- **Trojan Horse** → malicious hidden code
- **Trap Door (Backdoor)** → hidden access point
- **Buffer Overflow** → exploit program bugs
- **Virus** → self-replicating malicious code

System Threats

- **Worms** → self-replicating processes
- Consume system resources → system slowdown/crash

OPERATING SYSTEMS

Defenses against system attacks

Goal: Protect system from **unauthorized access and attacks**

Main techniques:

- **Cryptography**
- **Authentication**
- **Secure communication protocols**

Cryptography:

- Secures data using **encryption & decryption**
- Uses **keys** to protect communication

Provides:

- Confidentiality
- Integrity



OPERATING SYSTEMS

Defenses against system attacks

Authentication:

- Verifies identity of user/system
- Ensures:
 - Sender is genuine
 - Data is not tampered

Example:

- Passwords, OTP, multi-factor authentication



OPERATING SYSTEMS

Secure Communication (Protocols)

SSL / TLS:

- Provides **secure communication over internet**
- Used in:
 - HTTPS (web browsing)

SSH (Secure Shell):

- Secure remote login and file transfer
- Uses encryption for safe communication
- Runs on **port 22**

Overall:

- Cryptography → protects data
- Authentication → verifies identity
- Protocols → secure communication channel



OPERATING SYSTEMS

Implementing Security Defenses - Security Policy & Vulnerability Assessment



- Security implementation requires:
 - Policies
 - Monitoring
 - Prevention mechanisms

Key Idea:

- “Define rules + detect attacks + prevent damage”

Security Policy:

- Defines:
 - What is allowed
 - What is not allowed
- Guides system security decisions

Vulnerability Assessment:

- Identifies system weaknesses

Includes:

- Weak passwords
- Unauthorized programs
- Incorrect permissions
- Modified system files

OPERATING SYSTEMS

Implementing Security Defenses - Intrusion Detection & Virus Protection



Intrusion Detection:

- **IDS (Intrusion Detection System)**
 - Detects attacks and raises alerts
- **IPS (Intrusion Prevention System)**
 - Detects and blocks attacks

Virus Protection:

- Antivirus software:
 - Detects known viruses
 - Removes malicious code

Best method: **Prevention (safe usage)**

OPERATING SYSTEMS

Implementing Security Defenses - Logging, Auditing & Firewall

Logging & Auditing:

- Records system activities
- Helps detect:
 - Unauthorized access
 - Abnormal behavior

Firewall:

- Controls incoming & outgoing network traffic
- Allows or blocks connections based on rules





THANK YOU

Pavan A C

Department of Computer Science and Engineering

pavanac@pes.edu

OPERATING SYSTEMS

UE24CS242B



File Management

Pavan A C

Department of Computer Science & Engineering

OPERATING SYSTEMS



AWK and SED

Pavan A C

Department of Computer Science & Engineering

OPERATING SYSTEMS

Slides Credits for all PPTs of this course



- The slides/diagrams in this course are an **adaptation, combination,** and **enhancement** of material from the following resources and persons:
1. Slides of Operating System Concepts, Abraham Silberschatz, Peter Baer Galvin, Greg Gagne - 9th edition 2013 and some slides from 10th edition 2018
 2. Some conceptual text and diagram from Operating Systems - Internals and Design Principles, William Stallings, 9th edition 2018
 3. Some presentation transcripts from A. Frank – P. Weisberg
 4. Some conceptual text from Operating Systems: Three Easy Pieces, Remzi Arpaci-Dusseau, Andrea Arpaci Dusseau
 5. Internet source

OPERATING SYSTEMS

Text Processing Tools

Used for:

- Processing text files
- Data extraction & transformation

Common tools:

- **awk**
- **sed**



- Scripting language for:
 - **Pattern scanning & processing**
- Works line-by-line
- Used for:
 - Data filtering
 - Report generation

Working

- Reads file line by line
- Splits line into **fields**
- Applies:
 - Pattern → Action

Syntax:

- `awk 'pattern {action}' file`

Example:

- `awk '{print $1,$4}' employee.txt`

OPERATING SYSTEMS

AWK – Important Operations

- Field access → \$1, \$2, ...
- Conditions → if
- Built-in variables:
 - NR → number of lines
 - NF → number of fields



```
awk '{print $1,$4}' file.txt
```

Print selected columns

```
awk '/manager/ {print}' file.txt
```

Pattern matching

```
awk 'END {print NR}' file.txt
```

Count lines

- Stream editor for:
 - **Text transformation**
- Works in a **single pass**
- Used in pipelines

Syntax

`sed 'command' file`

Example:

`sed 's/hello/world/' file.txt`

OPERATING SYSTEMS

SED – Common Operations

Substitute → `s/old/new/`

Delete → `d`

Print → `p`

Insert / Append



OPERATING SYSTEMS

SED – Selected Examples

sed 's/hello/world/' file.txt

Replace text

sed '2d' file.txt

Delete line

sed -n '2p' file.txt

Print specific line

sed '4,6d' file.txt

Delete range



OPERATING SYSTEMS

AWK vs SED



Feature	AWK	SED
Type	Programming language	Stream editor
Focus	Data processing	Text editing
Works on	Fields	Lines
Complexity	Higher	Simpler



THANK YOU

Pavan A C

Department of Computer Science and Engineering

pavanac@pes.edu